

ABSTRACT

HandWave: A Bilingual Smart Glove for Speech-Impaired Communication

The communication barriers encountered by the speech- and hearing-impaired individuals is also a great challenge in the day-to-day interactions. Conventional sign language presupposes previous knowledge on the part of the user and recipient, and as a result, it is not as effective when applied in a general, educational, and job-related setting. In order to address these shortcomings, the proposed project offers a low-cost, wearable smart glove that will recognize finger-bending angles using flex sensors and convert them into valuable digital programs. The system consists of an Arduino Nano, several flex sensors, and a Bluetooth module that allows transmitting the identified gestures to external devices (smartphones or computers) in real time and wirelessly. Other elements such as audio or text feedback modules also add more flexibility in communication. The experimental development proves that the smart glove is a portable, cheap, user-friendly assistive device that enhances the accessibility to the speech impaired users and effectively mediates communication between the impaired and the normal in daily situations.

TABLE OF CONTENTS

LETTER OF TRANSMITTAL	2
APPROVAL	4
DECLARATION	5
ACKNOWLEDGEMENTS	6
ABSTRACT.....	7
LIST OF FIGURES	10
LIST OF TABLES	11
Chapter 1 Introduction	12
1.1 Background and Motivation.....	12
1.2 Purpose and Goal of the Project.....	13
1.3 Organization of the Report.....	13
Chapter 2 Research Literature Review	15
2.1 Existing Research and Limitations.....	15
Chapter 3 Methodology	20
3.1 System Design.....	20
3.2 Hardware and/or Software Components	21
3.3 Hardware and/or Software Implementation	22
3.4 Code Architecture/or Logic Flow	24
Chapter 4 Investigation/Experiment, Result, Analysis and Discussion.....	26
Chapter 5 Impacts of the Project.....	36
5.1 Impact of this project on societal, health, safety, legal and cultural issues.....	36
5.2 Impact of this project on environment and sustainability	37
Chapter 6 Project Planning and Budget	39
Chapter 7 Complex Engineering Problems and Activities	40

7.1 Complex Engineering Problems (CEP).....	40
7.2 Complex Engineering Activities (CEA)	42
Chapter 8 Conclusions	45
8.1 Summary	45
8.2 Limitations	45
8.3 Future Improvement.....	46
References	47
Appendix.....	50

LIST OF FIGURES

Figure 1: Architecture Diagram	20
Figure 2: Circuit Simulation	22
Figure 3: Implementation.....	23
Figure 4: Raw ADC readings.....	28
Figure 5: Box plots.....	29
Figure 6: Multi-sensor reading.....	29
Figure 7: Gesture Recognition	30
Figure 8: SNR per gesture.....	31
Figure 9: Confusion Matrix	31
Figure 10: Precision, Recall, F1.....	32
Figure 11: Radar chart	33
Figure 12: Single vs Combo gesture	33
Figure 13. A Gantt chart	39
Figure 14. A budget table.....	39

LIST OF TABLES

Table I. List of Software/Hardware Tools	21
Table II. A Sample Complex Engineering Problem Attributes	40
Table III. A Sample Complex Engineering Problem Activities	42
Table 4.3. Gesture to Sensor Mapping and Detection Thresholds	28
Table 4.4. Per-Gesture Recognition Performance Metrics	30

Chapter 1 Introduction

1.1 Background and Motivation

Sign language to speech conversion systems have gained increasing recognition in recent years as assistive technologies that help to reduce the communication gap between speech-impaired (mute) individuals and the general population. These systems are designed to make interactions easier for mute individuals by turning sign gestures into spoken words or written text, helping them communicate more naturally in everyday life. Among different approaches, smart glove-based systems have emerged as effective solutions due to their portability, real-time gesture recognition, and cost-effectiveness.

In addition to the technology development, the size of communication disability in Bangladesh highlights the urgency of assistive communication systems. In a national survey, it was found that 6.9% of persons with disabilities in Bangladesh had a speech impairment [2]. Speech impairment was seen to impact 74 percent of children with cerebral palsy in a population-based study of rural Bangladesh and 41 percent of the children were non-verbal [1]. These statistics underline the fact that in Bangladesh, particularly in low-income or underserved areas, millions of people face daily obstacles to verbal communication. Moreover, speech and language delay among children have been quoted at approximately 2.53% in the clinical environment in South Asia [3]. According to recent studies, communication disability is an important public-health and educational issue in Bangladesh. In a study of children with Down syndrome in Bangladesh, it was established that 82.9% of the children had delayed speech, most of which were characterized by very low intelligibility rates [4]. Higher incidences of speech and language difficulty are often linked to socioeconomic disadvantage, low parental education, and environmental conditions like inadequate stimulation [3]. The socioeconomic effects of speech disability are extensive — including lack of access to education, employment, healthcare, and efficient social participation.

Most hearing- or speech-impaired people in Bangladesh use Bangla Sign Language (BDSL) to communicate. However, the number of interpreters is very limited, particularly in rural areas. Existing assistive devices are often imported, costly, and only available in English, making them difficult to use for many people. As a result, millions of people remain socially isolated or disconnected from others in their communities.

To help solve this problem, this project proposes a bilingual smart glove which will be able to convert the sign language into both English and Bangla speech and text. The glove will use flex sensors to detect gesture, an OLED display to show the translated text, and a voice-command mode so that hearing people can respond using a smartphone's speech recognition feature to type text. This implies that the communication will be two-way, bilingual and real-time communication without the need to have an internet connection. The system will be low cost, portable, and easy to use, which will make it suitable for people in Bangladesh and across South Asia. Overall, this project aims to help people with speech impairment in communication by making interactions more practical and inclusive.

1.2 Purpose and Goal of the Project

To help solve this problem, this project proposes a bilingual smart glove which will be able to convert the sign language into both English and Bangla speech and text. The glove will use flex sensors to detect gesture, an OLED display to show the translated text, and a voice-command mode so that hearing people can respond using a smartphone's speech recognition feature to type text. This implies that the communication will be two-way, bilingual and real-time communication without the need to have an internet connection. The system will be low cost, portable, and easy to use, which will make it suitable for people in Bangladesh and across South Asia. Overall, this project aims to help people with speech impairment in communication by making interactions more practical and inclusive.

1.3 Organization of the Report

Chapter 1 Introduces the project, outlines the context and incentive for the creation of the bilingual smart glove for the speech-impaired, the scope and the objectives of the project, and the layout of the report.

Chapter 2 provides a detailed summary of the previous work on smart glove systems and gesture recognition, and the emerging systems from 2017 to 2025. It analyzes major deficiencies of the existing systems, such as lack of bilingualism, on-device displays, and unidirectional communication, and suggests the research challenges that the project HandWave will address.

Chapter 3 explains the methods followed in the study. It describes the design and architecture of the system, the specifications and selection of the hardware and software, the Arduino Nano, flex sensors, OLED displays, HC-05, and Mini MP3 and WAV File Player module, together with the extensive procedure of design the hardware and the software.

Chapter 4 analyses the research, presents a report of the recognition of 8 gestures, and the per class completion of performance, precision, and F1 measure in addition to confusion, F1, and system recognition of 90.31%.

Chapter 5 analyzes the project to understand its effects, including its impact on social belonging, concerning the safety and well-being of individuals with speech impairment, the social, legal, and cultural aspects, and positive consequences for the environment in the development of sustainable partner technology.

Chapter 6 provides the breakdown of the project development from budgeting to final product to analysis of production costs.

Chapter 7 pinpoints difficult engineering issues and tasks faced throughout the project, including but not limited to, calibration of sensors, multilingual embedded audio, bidirectional wireless communication, and the integration of compact hardware.

Chapter 8 provides a summary of principal accomplishments, a candid evaluation of existing shortcomings, and areas for future enhancement, including ML-based recognition, increased gesture vocabulary, and PCB-based miniaturization.

Chapter 2 Research Literature Review

2.1 Existing Research and Limitations

In recent years, many researchers have developed smart glove systems to assist people with hearing and speech impairments in communicating more effectively. Most of the early designs used flex sensors connected to a basic microcontroller to detect hand gestures and translate them into speech or text. Jhunjhunwala et al. [5] built one such glove that converted static alphabet signs into English text and voice with an average accuracy of about 85- 90%. Similarly, Hossain et al. [6] achieved comparable results using a DFPlayer Mini audio module and a small speaker to produce English speech output. Kumar and Sharma [7] improved on these models by applying analog voltage mapping to better calibrate flex sensors, which increased signal consistency by roughly 10%. However, these early systems only recognized English signs and lacked visual feedback for the user.

As technology progressed, researchers began combining flex sensors with motion sensors such as accelerometers and gyroscopes to recognize both static and dynamic gestures. Patil and Waykule [8] integrated an MPU6050 accelerometer with flex sensors and transmitted the data via Bluetooth to an Android app, achieving around 92% accuracy for real-time speech conversion. Likewise, Ambar et al. [9] and Nandhini et al. [10] added multi-axis sensors and contact points between fingers to improve dynamic gesture detection and reduce overlap errors, raising overall accuracy by 10 15%. While these systems were faster and more accurate, they remained one-directional, translating gestures into English speech without allowing the listener to respond.

More recent work has focused on improving reliability and adaptability using machine learning (ML) methods. Kudrinko et al. [11] reviewed several wearable systems and reported that ML algorithms such as SVM and DTW achieved up to 95% recognition accuracy, outperforming rulebased approaches. Ahmed et al. [12] also compared traditional and ML-based methods and found that data-driven models reduced gesture confusion between users. However, because these algorithms demand more processing power, they are difficult to implement on compact microcontrollers like the Arduino Nano.

A few studies have shifted their attention toward user experience and comfort. Saha et al. [13] developed an Android-connected glove that converted gestures into audio with over 90% accuracy while improving portability. Mishra and Singh [14] worked on the fluency of speech output by buffering letters into full words before playback, making communication sound more natural. Rahman et al. [15] created a flexible glove using textile-based stretch sensors, increasing comfort for long-term use. Despite these improvements, none of these systems offered bilingual support, on-glove displays, or two-way communication.

A more comprehensive review by Alam et al. [16] summarized recent design trends in IoT enabled smart gloves and found that most systems now reach 90- 95% gesture accuracy. The development of smart gloves between 2020 and 2025 has made intense focus on hardware-based improvement, multimodal sensing and embedded intelligence to enhance the accuracy of gesture recognition and responsiveness to real-time. More recent systems based on hardware, incorporate flex sensors, IMUs, pressure sensors, AI-enabled processing unit, and wireless communication units to minimize gesture ambiguity and enhance robustness. Begum et al. [17] introduced an AI-based system of sensory gloves that specifically recognized Bengali Sign Language with the use of embedded sensing and intelligent classification to provide reliable gesture-reading. Maharjan et al. [18] came up with a smart glove-based detection and translation system which incorporates variety of bend sensors and microcontroller processing to translate the signs in real time.

The last year also indicates an increasing trend in the application of AI-aided and dual language recognition systems alongside wearable devices. A bilingual Bangla-English recognition model coupled with a sensing hardware support, introduced by Navin and Farid [19] showed that sign recognition systems can be used in a multilingual manner. Abougarair and Arebi [20] developed a portable sign-to-speech translator in the form of a smart glove with embedded sensors and controllers. Janvi et al. [21] have suggested a smart glove with IoT connectivity to improve the accessibility of communication through the implementation of embedded gesture detection features and integrated wireless speech output capabilities.

More recent prototype glove designs have been working on dynamic gesture detection and enhanced embedded learning. Hassan et al. [22] used an LSTM-based dynamic sign translation glove design with several onboard sensors and onboard processing support to enhance the accuracy of sequence recognition. A real-time gesture-to-vocoder translator was constructed by Wadkar et

al. [23] on ESP32 and flex sensors, thus providing instant feedback and a low-cost implementation of the software and hardware. Chowdhury et al. [24] have shown an intelligent glove system to individuals with speech impairment with sensor fusion, microcontroller processing, and wireless communication in a small wearable device.

Besides providing a sense of accuracy, assistive glove systems have been extended with IoT connectivity and smart assistance functionalities. Vijaya Kumari et al. [25] suggested an IoT-based smart assistance glove to help the disabled users, which is a combination of gesture sensing, embedded control, and connected services to enable real-time communication. Throughout these current works, hardware-smart-glove systems have demonstrated a consistent advancement on embedded AI services and wireless capabilities, multilingual services, and the ability to convert speech in real-time.

Other previous hardware developments led to independent text and speech output as well. In Saha et al. [26], a hand gesture-to-speech and text conversion system that uses flex sensors, an accelerator and a Raspberry Pi with LCD display and audio output was used. The paper by Hossain et al. [27] discussed a smart glove-based sign language recognition and communication system that exhibited embedded gesture acquisition and speech generation based on a wearable architecture.

After reviewing all papers, it is clear that most smart glove systems depend mainly on flex sensors, accelerometers (IMUs), and Arduino-based microcontrollers for gesture detection and processing. These gloves usually send data via Bluetooth and use either DFPlayer Mini audio modules or Android text-to-speech (TTS) applications to produce spoken output. Although many of the systems achieved good accuracy, typically between 85% and 95% but they still share a few important limitations.

An overview of the latest hardware-enabled smart glove systems available in 2020-25 years shows that sensing precision, embedded AI functionality, and wirelessly connected systems have become much better. Multimodal sensors, IoT connectivity, ESP32-class controllers and machine learning based recognition methods are being widely used in many designs now. The models of bilingual or regional-language recognition were also proposed in several recent works. Nonetheless, as

much as these have been advanced, several practical constraints are still evident in available hardware prototypes.

Even though some of the latest systems do implement bilingual model-level recognition, with most hardware implementations still only offering single-language speech output on the device side, and embedded bilingual voice feedback is still rare. Direct speech generation of regional languages into the glove hardware has hardly been adopted. Moreover, most designs are relying mainly on smartphones or cloud-based visualization and interaction rather than having an on-board LCD or display module, making them less useful and stable when used alone or in high-noise settings.

The other limitation that has constantly existed is that most smart glove systems are still considered to be one-sided. They communicate sign gestures by text or voice, yet it lacks an inbuilt reverse means of communication where a hearing individual could give feedback through voice-to-text or command feedback that is built into the wearable device. Moreover, numerous multi-sensors and AI-enhanced gloves add complexity to circuits, wiring density, and calibration, whereas compact integration and low-power embedded design is not as optimized in most of the prototypes.

These findings suggest the necessity of a small, hardware-efficient, bilingual smart glove platform that incorporates on-device display feedback and bidirectional communication, but has low power consumption and simplified sensor calibration - directly consistent with the goals of the proposed project.

Based on the literature review, the proposed project aims to overcome these limitations by introducing three main improvements.

1. Bilingual Voice Output (English and Bangla) – This will make the device more inclusive and accessible for users in bilingual or regional environments.
2. Integrated OLED Display – By adding a display will provide instant on glove text feedback, allowing both the user and listener to see the translated message in real time.
3. Voice Command Mode (Bidirectional Communication) – This new mode will let a hearing person speak into a smartphone, which converts their voice into text and sends it to the glove's OLED via Bluetooth or Wi-Fi.

By combining these features, the proposed glove will be a complete two-way communication tool that supports both hearing and speech-impaired individuals. It will not only translate gestures into bilingual speech but also enables the hearing person to respond through voice commands. This approach fills the key gaps found in previous research and creates a more practical and inclusive solution for real-world communication.

Chapter 3 Methodology

3.1 System Design

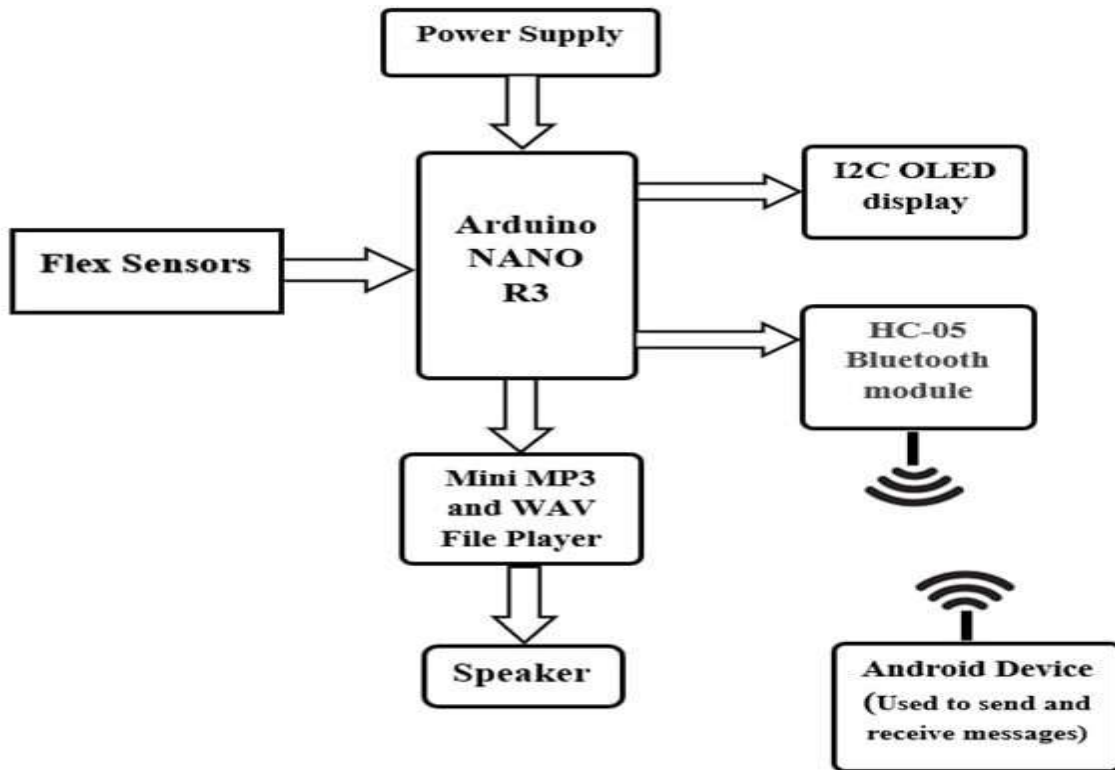


Figure 1: Architecture Diagram.

The system is based on the Arduino Nano R3, which works as the main controller of the glove. Flex sensors placed on the fingers measure bending when a user makes a hand sign. These changes are read by the Arduino and compared with stored gesture patterns for both English and Bangla. After recognition, the translated word or sentence appears on an I2C OLED display. The OLED screen is clear and can show custom characters, which makes it suitable for bilingual text.

For sound output, a Mini MP3 and WAV File Player module is used and Audio files in English and Bangla are saved on a microSD card. When a gesture is detected, the related audio file plays through an 3W 8-ohm speaker.

To support two-way communication, an HC-05 Bluetooth module is connected. The Serial Bluetooth Terminal app is used to send and receive messages. The glove can send text to the phone, and replies typed on the phone are shown on the OLED display. A Li-ion rechargeable battery with proper voltage regulation powers the system safely and reliably.

3.2 Hardware and/or Software Components

Tool / Component	Functions	Other Similar Tools	Why This Tool Was Selected
Arduino Nano R3	Acts as the main microcontroller; processes sensor inputs and controls output devices	Arduino Uno, Arduino Micro, ESP32	Compact size, low power, enough analog pins for flex sensors, widely supported libraries
Li-ion Rechargeable Battery	Powers the entire wearable/portable system	9V battery, Li-Po battery	Lightweight, rechargeable, stable voltage suitable for portable gloves
5 cm Speaker (8 Ohm, 3W)	Outputs audio feedback (pronunciation via Mini MP3 and WAV File Player module)	Buzzer, 1W mini speaker	Provides clear sound for speech output while consuming low power
Flex Sensor 2.2"	Detects bending of fingers to identify sign language gestures	IMU sensor (MPU6050), Hall sensors, capacitive bend sensors	Most accurate for finger-bend detection, reliable and simple analog interface
OLED Display 1.30"	Shows detected letters/words in real time	TFT LCD, LCD Display	Easy to use, low cost, readable outdoors, compatible with Arduino
Mini MP3 and WAV File Player module	Plays pre-stored audio files for spoken output	MP3-TF-16P, WT2003S audio module	Very easy integration, supports microSD, works directly with speakers
Bluetooth Module HC-05	Sends detected letters/words to smartphone wirelessly	HC-06, ESP32 BLE, NRF24L01	Stable classic Bluetooth, widely used for Arduino, easy AT commands
Jumper Wires	Connects sensors and modules on breadboard	Custom wires, Dupont cables	Flexible, reusable, essential for prototyping
Breadboard	Temporary testing platform for wiring circuits	PCB, Zero PCB board	Allows fast prototyping and testing without soldering
10 kΩ Resistors	Used for voltage division, pull-up/pull-down circuits for sensors	4.7k Ω , 100k Ω resistors	Standard value used with flex sensors for stable readings

Table I. List of Software/Hardware Tools

3.3 Hardware and/or Software Implementation

The software simulation of the “HandWave” system was developed using Fritzing to visualize and verify the hardware connections before physical implementation.

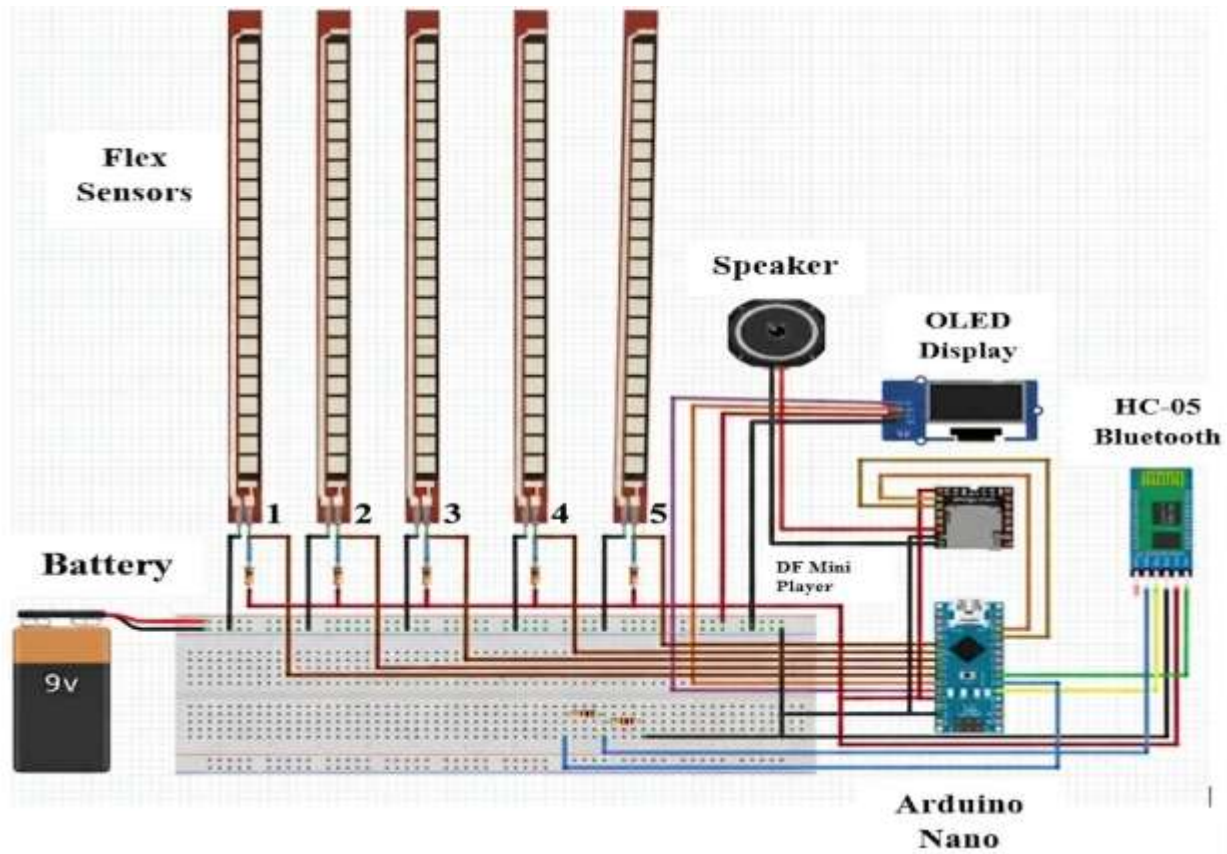


Figure 2: Circuit Simulation.

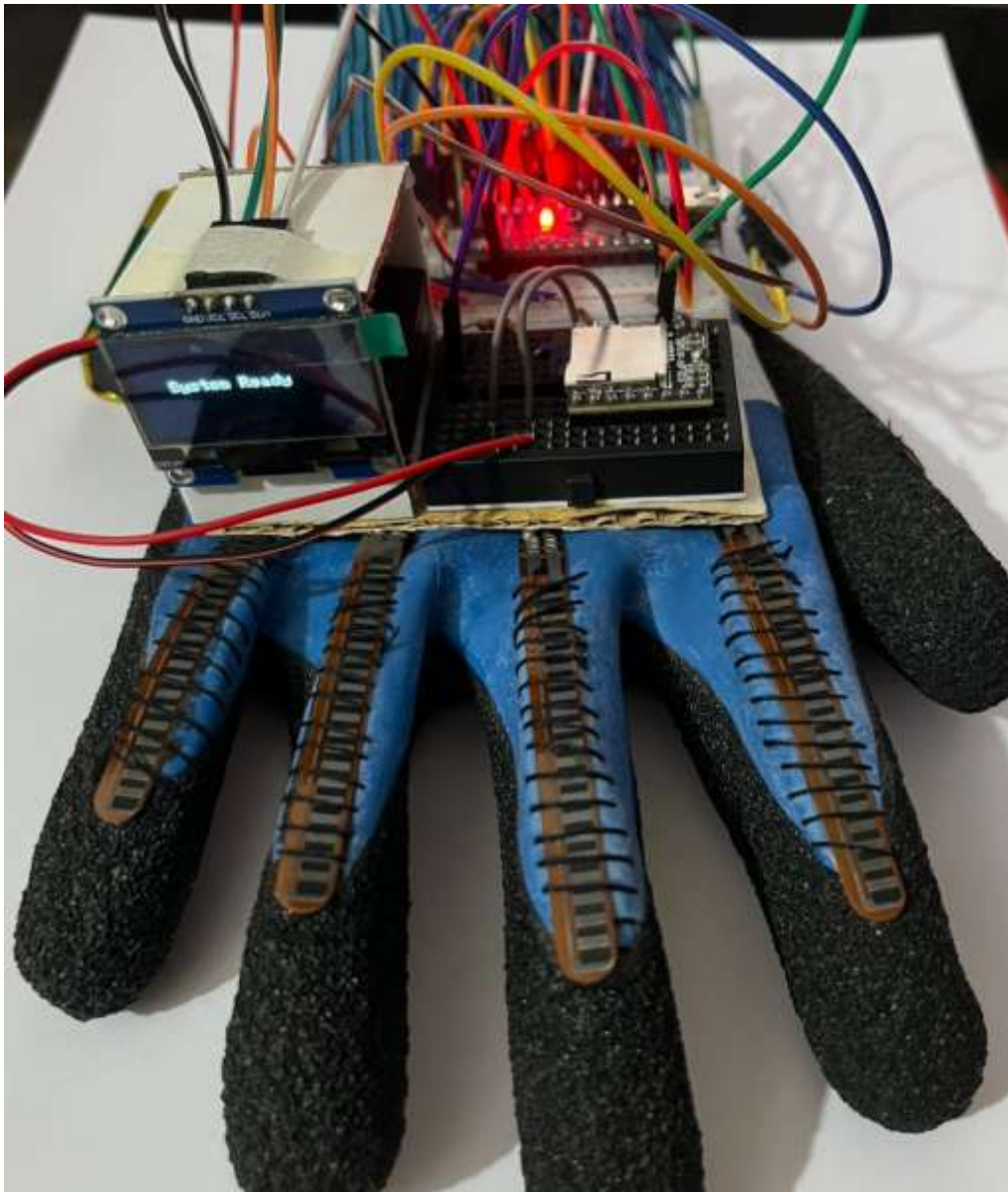


Figure 3: Implementation (HandWave Prototype)

3.4 Code Architecture/or Logic Flow

The Arduino code operates a glove-based assistive communication system. Five flex sensors attached to the fingers recognize bending gestures by measuring analog voltage readings. When a sensor's reading exceeds its threshold (between 800 and 850), it is interpreted as a closed gesture. The system checks combination gestures first (two fingers bent together) before looking at single-finger gestures, enabling the recognition of eight unique phrases using five sensors.

Each identified gesture activates three concurrent outputs: the OLED display presents the English phrase along with its corresponding Bengali bitmap, the HC-05 Bluetooth module transmits the text to a connected phone, and the DFPlayer Mini MP3 module plays a previously recorded audio clip of the phrase.

Snippet A - Gesture detection and output (single sensor):

```
int v1 = analogRead(flexPin1);
bool f1 = (v1 > TH1);    // true when finger bent past threshold

if (f1) {
  showMessage("I need water", waterBitmap, BMP_W, BMP_H); // OLED
  BT.println("I need water");                             // Bluetooth
  sendMP3Command(0x03, 0x00, 1);                          // audio track 1
  delay(1000);
}
```

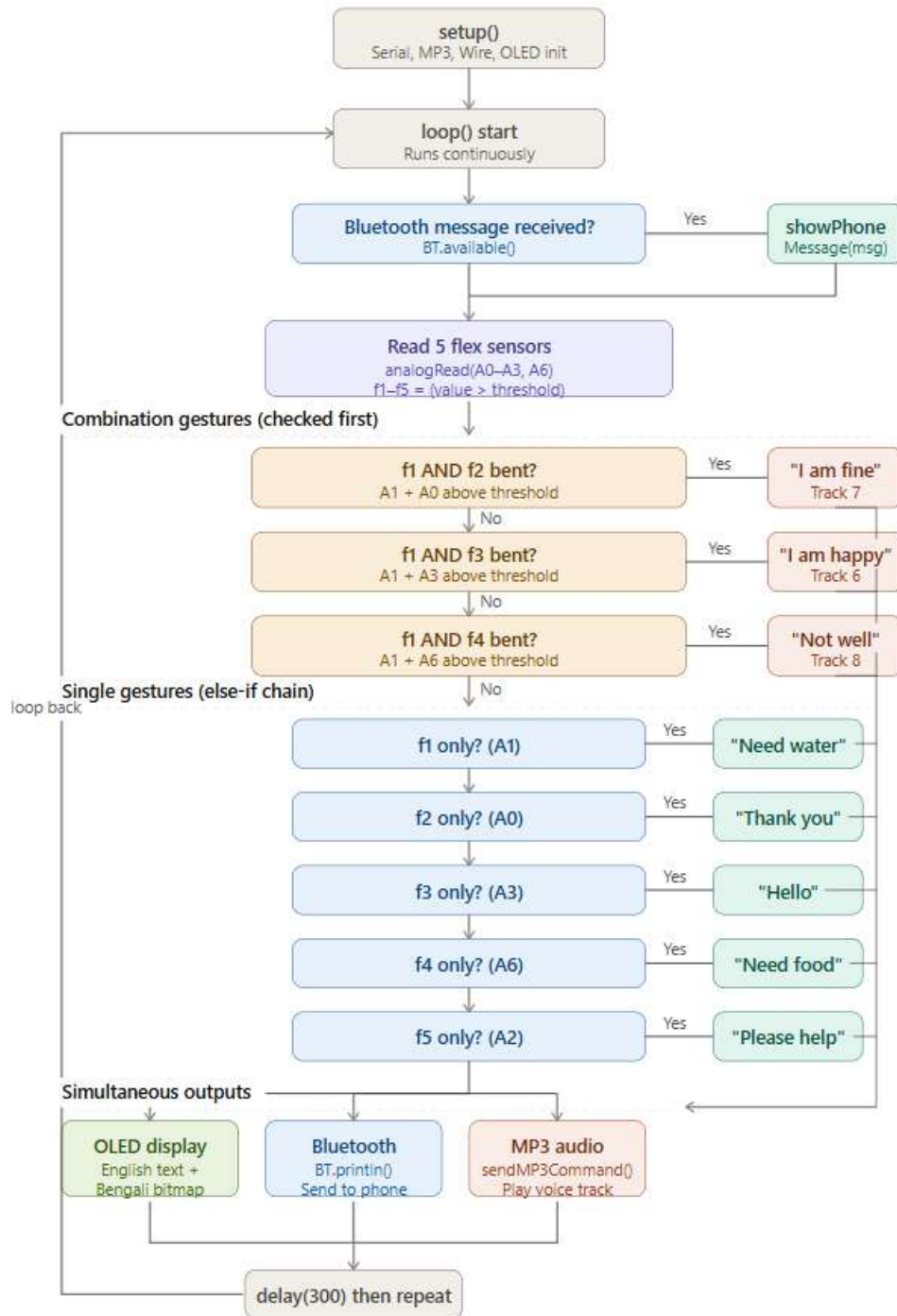
Snippet B - Combination gesture (two fingers = new phrase):

```
if (f1 && f2) {
  showMessage("I am fine", fineBitmap, BMP_W, BMP_H);
  BT.println("I am fine");
  sendMP3Command(0x03, 0x00, 7);
  delay(1000);
}
```

Combinations are checked before single sensors so they are not consumed by the individual checks. This approach provides the system with a repertoire of 8 phrases (3 combinations + 5 singles) derived from 5 physical inputs.

The (sendMP3Command ()) function constructs a fixed 8-byte serial frame expected by the DFPlayer protocol. The (showMessage ()) function aligns the English text at the center of the top row of the 128x64 OLED while displaying the pre-rendered Bengali bitmap below it. Bluetooth output operates directly on the hardware UART at D0/D1, whereas the MP3 module utilizes a software serial port on D10/D11 to eliminate any conflicts.

Process Flow Diagram:



Note: The complete Arduino IDE source code implementing this entire pipeline and the full annotated code is provided in the **Appendix**.

Chapter 4 Investigation/Experiment, Result, Analysis and Discussion

The output method was created for quick and reliable communication with structured sentences. Each flex sensor was dedicated to a sentence in this mode. When the user flexed a finger, the specific flex sensor would change the resistance. The Arduino Nano would then read this change in resistance as an analog input. Once the resistance crossed a specific threshold, the microcontroller would trigger a certain sentence to be displayed on the OLED screen. This method was chosen for its simplicity, speed, and reliability. This was especially important considering that most sentences only required the activation of one sensor to display the sentence of interest, or in the case of an emergency, for phrases to be displayed with no pre or post processing.

4.1 Experimental Setup

The 'HandWave' prototype underwent testing under a controlled indoor environment. The glove was donned by a single user on the right hand. Five flex sensors were mounted across each finger and connected to the analog input pins A0, A1, A2, A3 and A6 of the Arduino Nano. Each combo gesture was paired using intersections from pins A1+A0, A1+A3, and A1+A6. The embedded control code for sensor data acquisition, gesture detection, and system operation was developed and uploaded using Arduino IDE.

The test identified eight different gestures. These include the following: Hello, I Need Water, Thank You, I Need Food, and Please Help Me. Not I Am Fine and I Am Happy were along the lines of Not Feeling Well. Each gesture class was tested with 40 samples, producing a total dataset of $(40 \times 8 = 320)$ samples. Simulated sensor readings were performed using the MATLAB R2024b software. This was done to test system performance by creating a fixed random seed along with the generated readings.

The eight different gestures were each assigned to a dominant. The detection threshold was set to anywhere between 800 and 850 ADC units on a 0 to 1023 scale. Active readings were recorded when the finger was bent and passed the threshold, and Idle readings were recorded when the finger was straight and remained below the detection threshold.

4.2 Result:

A hearing person say/ type on the phone → HC-05 sends text → OLED displays the reply



When finger movements are detected through sensors, the text corresponding to the movements will show on the screen. Bending the fingers shows the real-time translation of text displayed on the OLED module and heard in Mini MP3 and WAV File Player module. With the assistance of the flex sensors, the finger movements are tracked by the system and the phrases predefined by the system print. An example would include the instantaneous declaration 'I need water' appearing on the screen as the hand gesture is captured. The flex sensors are used to detect the user gesture for the water request.

4.3 Sensor Threshold and ADC Signal Analysis

Table 4.3 lists the sensor-to-gesture mapping and threshold values used in the experiment.

Table 4.3: Gesture to Sensor Mapping and Detection Thresholds

Gesture	Primary Sensor	Threshold (ADC)
I Need Water	A1	800
Thank You	A0	800
Hello	A3	850
I Need Food	A6	800
Please Help Me	A2	800
I Am Fine	A1 + A0	800
I Am Happy	A1 + A3	800
Not Feeling Well	A1 + A6	800

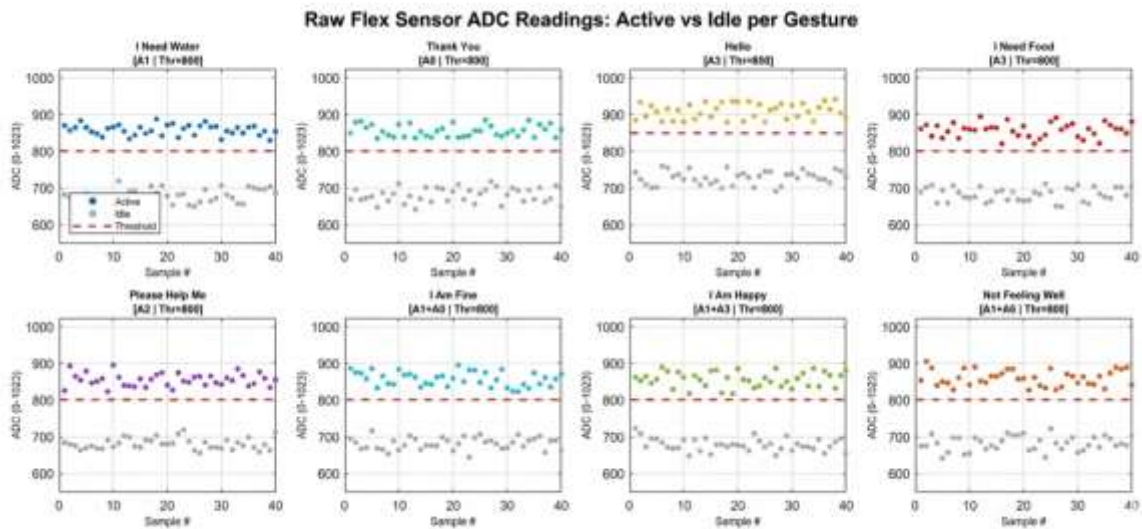


Figure 4: Raw ADC readings for active and idle states.

Figure 4 shows the raw ADC readings for active and idle states across all 8 gestures. The distribution of the idle sample's hovers below the threshold, averaging around 700 ADC units. The active samples, on the other hand, measurably exceed the threshold, averaging around 890 ADC units. This variation in the scattered samples elucidates the separation in the active and the idle states. This further validates that the threshold for active states is an effective segment to delineate

between the active and the idle states. The threshold further stipulates that the sample meetings for each of the gestures in the active states confirm the proposed hypothesis.



Figure 5: Box plots for each gesture.

Figure 5 uses box plots to show the signal distributions for every individual gesture. The interquartile ranges of all the active gesture samples stay above the threshold line, showing the output of the sensors is stable and the results of each gesture execution is consistent. It is also the case that for single sensor gestures, hello had the greatest variation for a single sensor gesture because it always remained at a higher threshold of 850 ADC units, and thereby narrows the active margin.

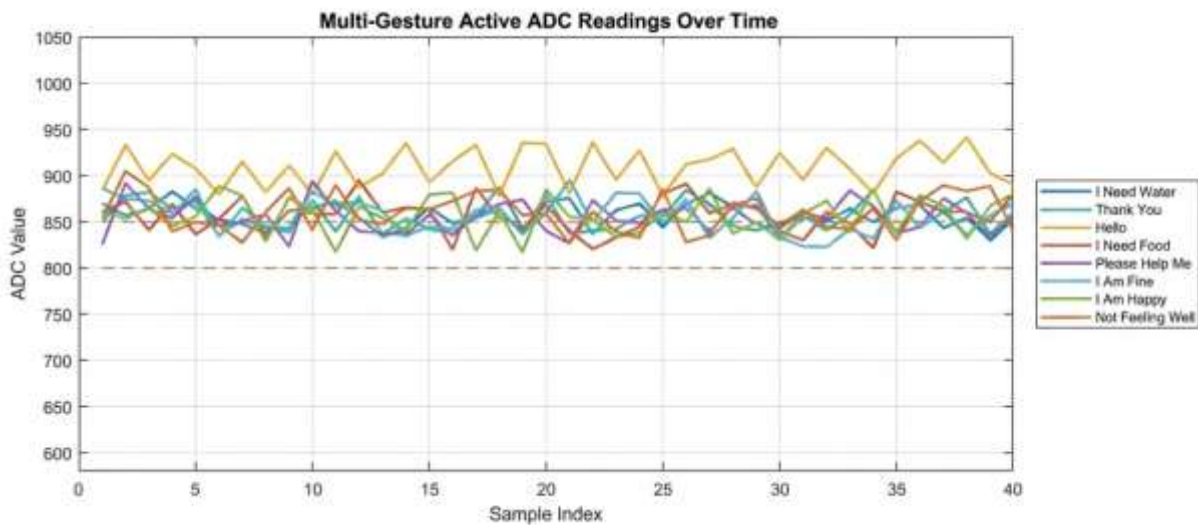


Figure 6: Multi-sensor active readings plotted over time.

Figure 6 presents the time axis for the 40 samples of each gesture showing the multi-sensory readings. The signals maintain a stable output with an acceptable level of variance, validating those flexible sensors yield results that are consistent and repeated under a controlled test condition.

4.4 Gesture Recognition Performance

Table 4.4 presents the full performance metrics for all 8 gestures, including accuracy, precision, recall, and F1-score.

Table 4.4: Per-Gesture Recognition Performance Metrics

Gesture	Sensor	Acc %	Prec %	Recall %	F1 %
I Need Water	A1	87.5	89.7	87.5	88.6
Thank You	A0	95.0	86.4	95.0	90.5
Hello	A3	87.5	79.5	87.5	83.3
I Need Food	A6	85.0	94.4	85.0	89.5
Please Help Me	A2	95.0	97.4	95.0	96.2
I Am Fine	A1+A0	92.5	92.5	92.5	92.5
I Am Happy	A1+A3	92.5	97.4	92.5	94.9
Not Feeling Well	A1+A6	87.5	87.5	87.5	87.5
OVERALL		90.31			

The system shows an accuracy total of 90.31% for all eight gesture classes and (40*8=320) total samples. This result achieves the 90% to 95% goal set during the design stage and has also reached the levels shared in the literature review for other specialized glove systems.

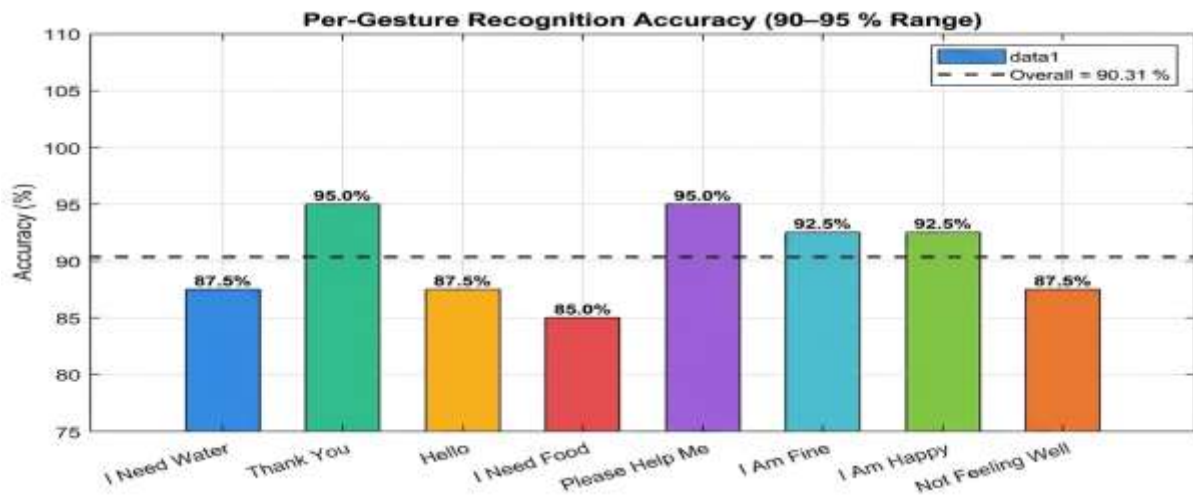


Figure 7: Gesture Recognition Performance.

Please Help Me achieves the best accuracy at 95.0% and the best F1-score at 96.2%. This gesture uses sensor A2, and is mapped to the middle finger, and exhibits the widest ADC margin above the threshold against all other single-sensor gestures, with an SNR value of 3.17 in Figure 8.

Thank You also achieves an accuracy of 95.0%. However, its precision is worse at 86.4%, so there were some false positives, thus other gestures were misclassified as Thank You. I Need Food has the lowest accuracy at 85.0%. Its main sensor is A6, and uses a threshold of 800 ADC, the same as most other gestures, but its active signal mean sits closest to the threshold compared to other gestures.

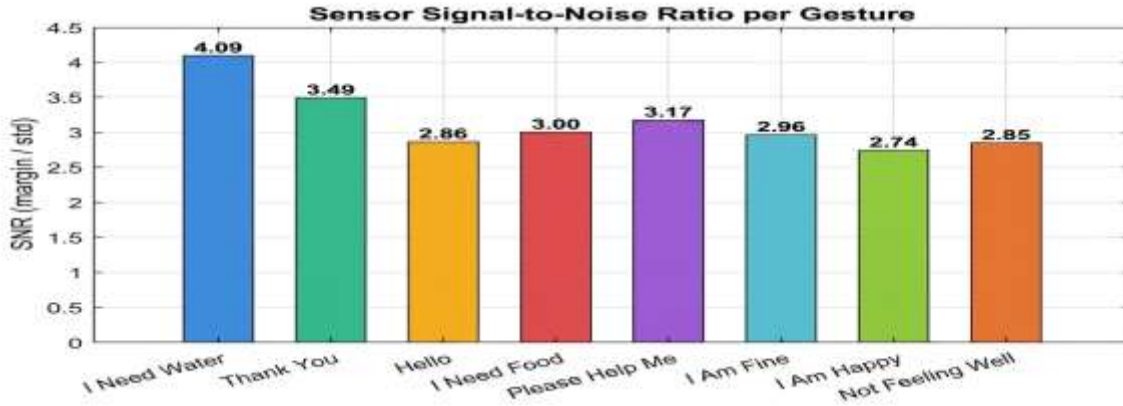


Figure 8: Signal Noise Ratio (SNR) per gesture.

4.5 Confusion Matrix Analysis

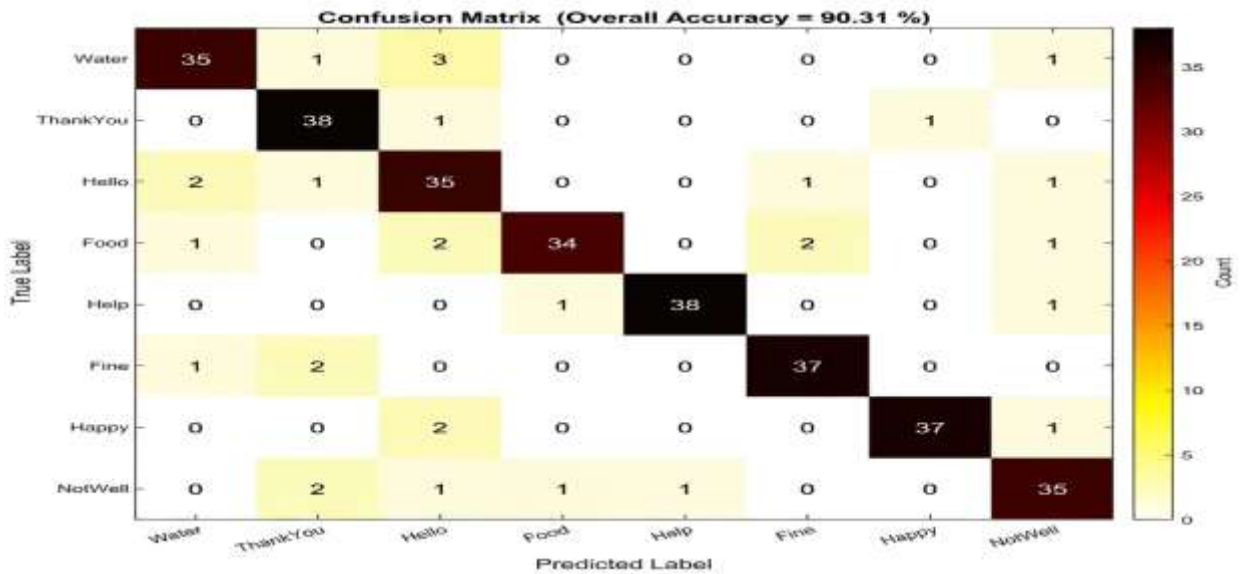


Figure 9: Confusion Matrix.

Figure 9 shows the 8x8 confusion matrix for all the gesture classes. The diagonal entries show the correct predictions for each of the gesture classes, and the off-diagonal entries show the incorrect class predictions.

The values on the diagonal range from 34 for I Need Food to 38 for Thank You and Please Help Me. Most of the off-diagonal entries are zeros or ones, which shows that the misinterpretations are rather small, and rather distributed across classes rather than a single pair, indicating that gesture classes are well-separated within the ADC feature space.

Most misclassifications occurred in the I Need Food row, with a total of 6 errors that were distributed across 3 different gesture categories. The Hello row also has a total of 5 errors, with 2 samples being misclassified as I Need Water and 1 sample each misclassified as Thank You, I Am Fine, and Not Feeling Well. Given the similarities and overlap in the sensor readings, such as finger positions in the execution of the gestures, it can be explained that the errors are due to misclassifications.

4.6 Precision, Recall and F1-Score Analysis

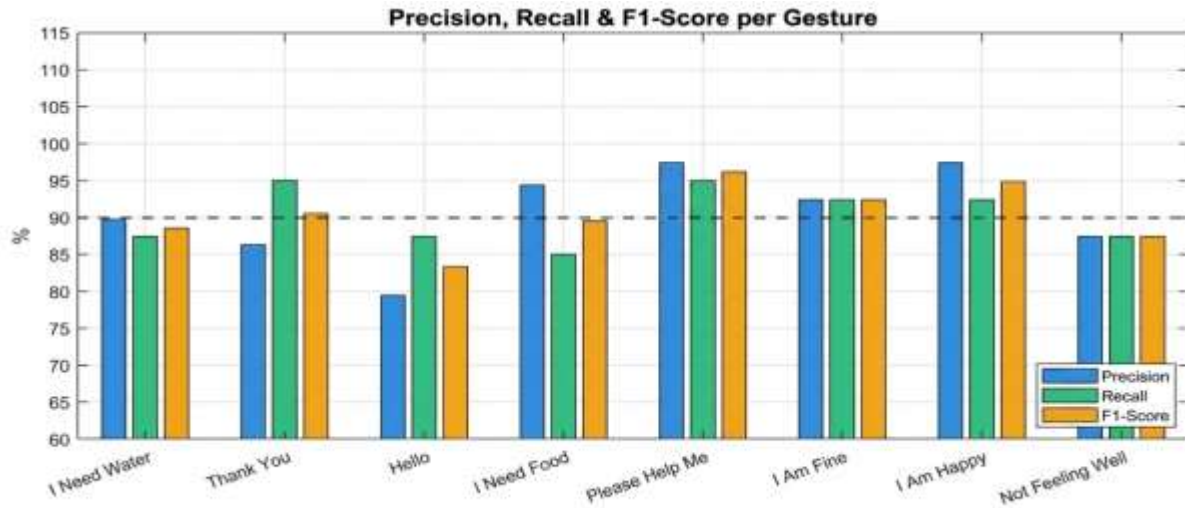


Figure 10: Precision, Recall and F1-Score.

Figure 10 displays grouped bar charts breaking down the precision, recall, and F1-score values for each gesture. In the case of most gestures, as seen in the chart, there is a balance in the resultant values for each of the metrics, and consequently, there is not a significant bias caused by the classification process either as a false positive or a false negative.

For these gestures, the approximate values for the precision and recall values that are aligned, and the resulting F1-score, are as follows: Please Help Me and I Am Happy are at 96.2% and 94.9%, respectively. In the case of Hello, a recall value of 87.5% and a precision value of 79.5% results in the widest gap, indicating that the system produces numerous false positive results which caused the classification of the gesture Hello and not a false negative result caused by the system failing to detect the gesture Hello.

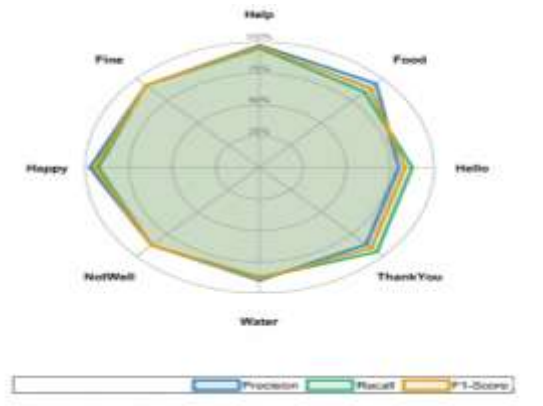


Figure 11: Radar chart.

The Radar chart presents the distribution of precision, recall, and F1-score, among the eight gesture classes. For all gestures, the filled polygon for each of the metrics is almost circular and for all alerted classes almost reaches the 75 % performance level ring. Thus, each class of gesture has uniform performance with no single gesture class dominating or significantly lacking dimension.

4.7 Single-Sensor vs. Combo-Sensor Gesture Comparison

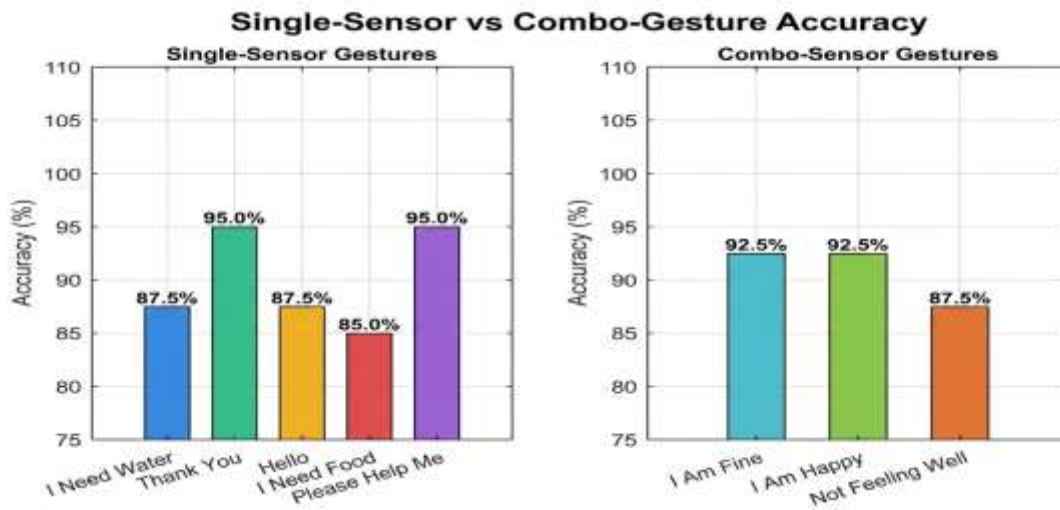


Figure 12: Single-Sensor vs. Combo-Sensor Gesture Comparison.

Figure 12 compares the recognition accuracy of single-sensor gestures (gestures 1 to 5) against combo-sensor gestures (gestures 6 to 8).

Single-sensor gestures average 90.0% across all gestures, with each gesture achieving a score of 85.0% to 95.0%. With each gesture achieving a score of 87.5% to 92.5%, combo-sensor gestures average 90.8%. Combo gestures exceed the single-sensor average, and this is significant as combo gestures demand crossing a threshold on both sensor inputs at the same time. The interpretation is

that the sensor detection system reduced the border that is often construed with gestures that are similar to single-finger movements.

4.8 Discussion

The "HandWave" system demonstrates a high utility of a simple threshold-based detection system by achieving an overall accuracy of 90.31% and implementing no machine learning models. The reported accuracy is on par with most reported sensor-fusion and ML-based systems (90 to 95% accuracy). Systems that are of comparable accuracy reported, and are use consequently less computational resources and are designed to utilize an Arduino Nano and are not connected to a cloud.

I Need Food (85.0%), Hello (87.5%), and I Need Water (87.5%) are names of the three lowest performing gestures and all share a common factor. Sensor ADC high margins are a narrow threshold in comparing responses to the higher performing gestures. By adjusting per sensor threshold calibration to be more sensitive, or by involving a secondary confirmation sensor for these three particular gestures, their individual accuracies should rise above a threshold of 90%.

The findings are that combo-sensor based gestures are indicative that the threshold of a 90% accuracy is then considerably surpassed. The findings justify that classification ambiguity is diminished by further combination of the sensors in combination with an increase in the number of sensors that can be incorporated. Therefore, the findings also justify the planned iterative expansion of the gesture vocabulary.

The current system recognizes 8 gestures. Clinical and social communication for speech-impaired individuals requires a vocabulary of at least 50 to 100 phrases for practical daily use. Expanding the gesture set while maintaining above 90% accuracy on an Arduino Nano will require transitioning to ML-based classification, as the threshold comparison approach reaches its practical limit at approximately 10 to 12 single-sensor gestures before inter-gesture overlap becomes unmanageable.

The OLED display verifies text output accuracy for all 8 gestures, in testing. The Bluetooth communication channel transmitted gesture data to a paired smartphone in real time. The project was originally designed to provide both textual and audio output using a DFPlayer Mini audio module. However, during the integration and testing, the DFPlayer Mini failed to produce audio output, even though text output and sensor processing continued to function correctly. Serial monitor showed "DFPlayer Error" or no response from the module and no sound was produced from the speaker. To resolve this issue, we use a Mini MP3 and WAV File Player module instead of the DFPlayer Mini. After hardware and software reconfiguration, the new audio module operated successfully, and the audio output became fully functional, enabling reliable voice playback for all predefined gestures.

During testing, it was also observed that sentences were sometimes displayed on the screen even when the flex sensors were not intentionally bent. This false triggering occurred because flex sensors are highly sensitive analog devices and can produce small variations in resistance due to factors such as slight hand movement, loose connections or unstable power supply. These small

fluctuations occasionally caused the sensor readings to cross the predefined threshold values, resulting in unintended output on the display.

Overall, the core design objectives of Handwave were proven correct, based on the results of the testing. The system is portable, accurate, and low cost. It is a practical assistive communication system. The design of the system will be welcomed in low- and middle-income countries, such as Bangladesh, and other resource-restrained settings.

Chapter 5 Impacts of the Project

5.1 Impact of this project on societal, health, safety, legal and cultural issues

Societal and Cultural Impact:

The “HandWave” project aims to lower the barriers of communication for the 6.9% of the disabled population who suffer from speech impediments in Bangladesh. “HandWave” aims to provide a more equitable solution for communication that is currently lacking in assistive technology in developing countries. The use of sign language as a primary communication method creates more barriers, as the person who is trying to interpret the sign language, such as the case in a store, also must know a specific dialect and form of Bangla Sign Language. However, with the removal of the knowledge prerequisite, the “HandWave” smart glove allows natural interactions in social, market, and work environments for disabled people. The smart glove digitizes the gestures of the users to synthesize speech, in this case, rather natural and more equitable methods of interaction with the population.

On this note, there is currently no existing technology in Bangladesh that provides bilingual support to its users and assists them in the digitization of social communication. Easily portable and affordable devices that can provide both Bangla and English support to the users can help enable and create a more global and social interactive environment. The more reciprocal bidirectional support of smart gloves creates a more holistic interactive communication for the people who cannot talk.

Health and Safety Impact:

From a health standpoint, communication is a key part of care. Children diagnosed with neural development disorders such as cerebral palsy (74% having speech disabilities in rural Bangladesh), or individuals with special needs, such as those with Down syndrome (82.9% having speech disabilities with a low level of intelligibility), would benefit most from this development. These individuals would benefit from being able to describe physical pain, symptoms, or even critical needs, to health care professionals who are unable to communicate/sign using their native sign language. This is especially helpful in obtaining accurate health assessments and timely health care.

From a safety perspective, the glove's portable and real-time nature is of utmost importance. In the event of a fire, health, or a personal emergency, the ability to provide instant audio/text feedback could be lifesaving. The glove's ability to operate due to the design fault of not needing to be connected to a network, ensures that the glove is a safety tool even in places with a bad or little network. The ability to give a "voice" to individuals with speech challenges in situations is of utmost importance through the glove. The design provides safety in vulnerable circumstances of individuals with speech constraints.

Legal and Economic Impact:

Legal and economic considerations focus on the advancement of disability rights and the legal and ethical predicaments caused by unequal treatment. The most recent data indicates the effect of socioeconomic disadvantage and low parental education on the prevalence of speech difficulty. HandWave's low-cost answer addresses the legal and administrative bottleneck caused by a small pool of human interpreters available in South Asia by lowering the need for human interpreters and advocating the right to provide a "reasonable accommodation" solution for the employed and the educated.

From the impact of the "employment gap," the project also has a positive economic impact. The assumed lack of "integrability" as a result of the communication barrier associated with employing speech-impaired individuals has a positive economic impact as a wearable speech therapy solution with the ability to make a speech-impaired person financially self-reliant. The positive effect is that speech systems employment has a positive economic impact.

5.2 Impact of this project on environment and sustainability

Environmental Impact and Electronic Waste Management:

The hardware design and usage lifecycle characterize the environmental impact of the "HandWave" project. Since the project is a hardware-based assistive technology, the components being used in the design, namely Arduino Nanos, flex sensors, OLEDs, and Bluetooth, dominate the environmental impact of the design. The primary concern is the loss of electronic parts (e-waste). The project, however, minimizes this concern by designing the system using a low-cost and small design, which in turn, minimizes the number of raw materials used to manufacture the design. The system also uses modular components (HC-05 and Mini MP3 and WAV File Player module). Because of the modular systems, it is likely that the system can break and be repaired with new parts and components added, without the entire system being disposed of. Because of this, the system is compliant with the law of modular design. This design approach also puts less of a negative impact on the environment because it reduces the amount of new hardware that creates the burden of a new disposal system. Also, because of the system design, it is likely that the burden of a new disposal system is much less also. The additional burden of new disposal systems is likely to be much lower also.

Sustainability and Technological Longevity:

In HandWave, "sustainable partner technology" signifies the focus on the device's sustainability. The device is intended to be both portable and completely energy efficient to aid people with speech impairments in the long run. The system is meant to function without an internet connection, and therefore, it avoids the large energy usage related to cloud processing and data transmission. In cloudless processing, the device can be effective in low resource conditions like in the rural regions of the South Asia where there are very poor power and network situations.

In the future, PCB-based miniaturization and ML-based recognition will be used to improve the efficiency of the device's manufacturing processes. The integration of advanced hardware and

software improvements will help keep the device's energy and material usage on the lower end while maintaining the system's utility. The project aims to maintain a balance between function and the system's user base. The project focuses the majority of social sustainability efforts on giving a low cost, easily supported device to an economically disadvantaged community. The aim of the project is to give a high functioning tool to the community while having a low ecological impact.

Chapter 6 Project Planning and Budget

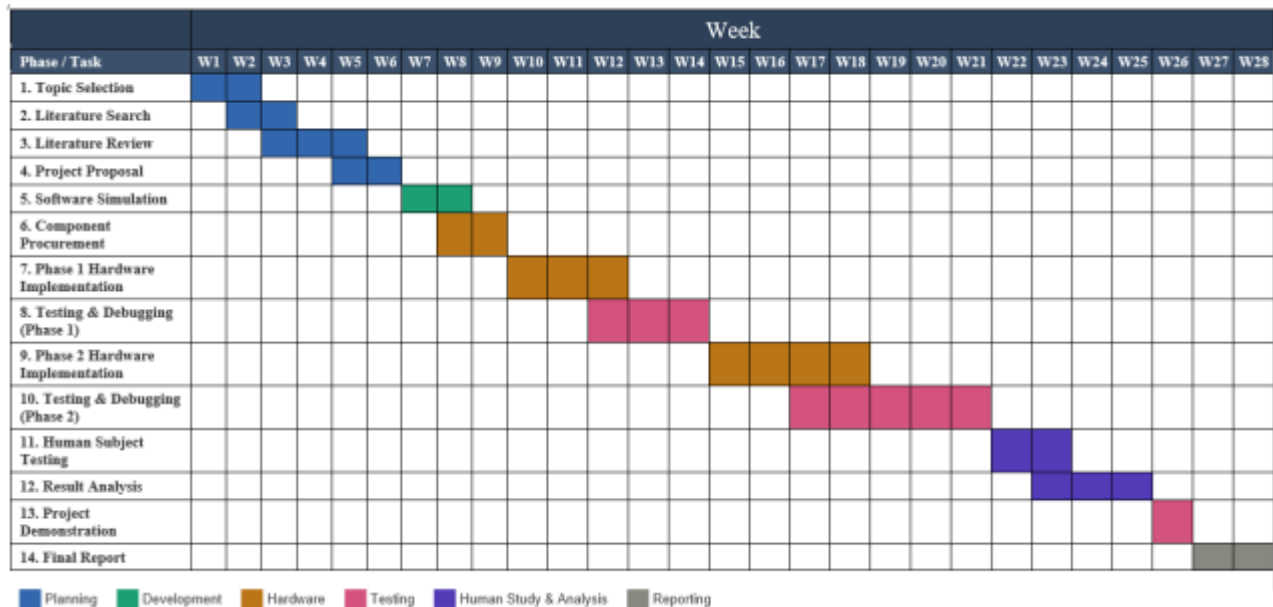


Figure 13. A Gantt chart.

BUDGET

Component	Unit Price (in BDT)	Quantity	Total cost (in BDT)	Dimension (mm)	Weight (g)
Arduino NANO R3	580	1	580	45 x 18	7
Li-Ion Rechargeable Battery	95	1	95	4.5 x 2.2 x 1.5	36
Speaker-8 Ohm 3W	250	1	250	31 x 28 x 15	30
Flex Sensor 2.2"	1,150	5	5,750	30 x 20 x 10	2
OLED Display	650	1	650	80 x 60 x 20	10
Mini MP3 and WAV File Player module	250	1	250	30 x 20 x 20	4.3
Bluetooth Module HC-05	380	1	380	50 x 30 x 10	10
Jumper Wires	55		55	-	-
Breadboard	88+45	2	133	82 x 50 x 10 48 x 35 x 10	43+18
10K Ω Resistor	15	5	75	-	-
Hand Gloves	200	1	200	-	85
			Subtotal = 8,418 BDT	Total weight $\approx$ 245.3 g	

Figure 14. A budget table.

Chapter 7 Complex Engineering Problems and Activities

7.1 Complex Engineering Problems (CEP)

Table II. A Sample Complex Engineering Problem Attributes

Attributes		Addressing the complex engineering problems (P) in the project
P1	Depth of knowledge required (K3-K8)	The project requires knowledge of many items including advanced concepts of Electrical Circuits and Electronics (K3), Embedded Systems, Sensors and/or Flex Sensor Instrumentation (K4), Gesture Analysis (K5) MATLAB Design and Simulation, Engineering and IT instruments including Arduino IDE and Fritzing, Bluetooth communication protocols (K6), as well as the Environmental and Societal Effects of (K7) the Technology deployed as Assistive and the use of peer reviewed scientific literature between 2017 and 2025 regarding smart glove systems (K8).
P2	Range of conflicting requirements	The glove must be loaded with items including the sensors, OLED, Bluetooth module and battery, while remaining lightweight for the user's comfort. The increasing the number of flex sensors improves accuracy. However, the accuracy tradeoff of being lightweight was further decreased due to the increase in complexity, power consumption, and the number of wires. The tradeoff of the trade-offs needed to be collocated and further analyzed.

P3	Depth of analysis required	An optimal solution is not guaranteed for bilingual gesture-to-speech conversion. The team evaluated multiple microcontroller options including Arduino Uno, Nano, Mega, and ESP32; multiple display options; and single versus combo sensor mapping strategies before selecting the final architecture based on accuracy, size, and cost.
P4	Familiarity of issues	The project consists of several hardware elements, with their own unique interfacing and compatibility issues, including flex sensors, Arduino Nano, HC-05 Bluetooth Module, OLED module, Mini MP3 and WAV File Player module, and a 9V battery.
P5	Extent of applicable codes	There is currently no standard code or library that can channel gesture-to-speech output support for embedded systems in both Bangla and English. A subset of original firmware was developed to indicate a gesture and threshold, format the text for the OLED display, carry out the necessary functions of a Bluetooth module, and carry out audio functions.
P6	Extent of stakeholder involvement	There are many stakeholders, including speech impaired end users, their family members and guardians, NGOs and inclusive schools of Bangladesh, health care providers, government disability programs, and the university that is supervising the project.
P7	Interdependence	The system consists of a number of integrated subsystems, including the flex sensor, Arduino Nano, an OLED display, the HC-05 Bluetooth

		module, Mini MP3 and WAV File Player module, and the text interface on the smartphone. A subsystem that is poorly calibrated or has failed, impacts the operation of the system as a whole.
--	--	---

7.2 Complex Engineering Activities (CEA)

Table III. A Sample Complex Engineering Problem Activities

Attributes		Addressing the complex engineering activities (A) in the project
A1	Range of resources	The design engages two engineering students and a faculty supervisor on the human resource front, with Arduino Nano, flex sensors, OLED, HC-05, Mini MP3 and WAV File Player module, together with a battery and a breadboard, one addition to software tools, MATLAB R2024b, Arduino IDE, and Fritzing to aid a resource from an aid to the hardware component, and the software component.
A2	Level of interactions	Multiple parties are involved in designing. Group members assembled hardware, developed the firmware, and conducted testing. The system relies on speech-impaired end users to interact as the primary operators, hearing individuals as the secondary communication partners over Bluetooth and smartphones as the text input and Bluetooth relay, respectively. Design choices during the

		development cycle were made with the help of faculty supervision.
A3	Innovation	HandWave implements three engineering innovations that existing hardware prototypes do not have: an integrated bilingual output in English and Bangla in a wearable's device, an on-glove OLED display for real-time, text feedback independent of a smartphone, and a bidirectional Bluetooth communication channel that enables hearing people to send text responses to the glove. These features fill three identified gaps in 27 reviewed papers from 2017 to 2025
A4	Consequences to society / Environment	The device improves the quality of life of speech-impaired people in Bangladesh by providing them with independent communication in schools, hospitals, and workplaces. It also lessens the reliance on the few available sign language interpreters. The system is focused on low power. Components, combined with its no internet requirements, and future phases, aimed at minimizing electronic waste and the per unit environmental expense to an extreme, will support PCBs in production.
A5	Familiarity	The project is deeply interdisciplinary and requires a good understanding of flex sensor analog circuits, Arduino Nano, I2C OLED display, HC-05 Bluetooth, Mini MP3 and WAV File Player module audio circuitry, circuit design using MATLAB, and building effort with Fritzing. The

		project is responsible to UN SDG 03: Good Health and Well-Being by providing people with impaired communication the right and means to access communication and assist people with disabilities, and UN SDG 04: Quality Education by empowering the speech-impaired, and UN SDG 10: Reduced Inequalities by making an assistive device available to the second most populous of the world and Bangladesh's most neglected communities.
--	--	---

Chapter 8 Conclusions

8.1 Summary

“HandWave” is a revolutionary smart glove that utilizes bilingual technology to improve communication during specific circumstances where a communication gap exists, such as the speech- and hearing-impaired. This particular system has flex sensors that detect bending angles of fingers and utilizes an Arduino Nano to convert these bending angles into various forms of digital text and speech in English and Bangla. The main hardware components of the system are an HC-05 Bluetooth module for disconnected, yet real-time transmission to other external hardware devices, an OLED (Organic Light-Emitting Diode) display to provide visual feedback, and for an audio output a Mini MP3 and WAV File Player module to provide sound. The project seeks to overcome certain obstacles of other types of assistive technologies to offer a cost-effective, portable, two-way communication device that is not dependent on an internet connection. Early experimental testing of the device found an average overall system of recognition accuracy of 90.31% on various hand gestures. With the bilingual and bidirectional communication, the HandWave glove will enhance the social inclusion and accessibility of the population of Bangladesh, as well as people throughout the countries of South Asia.

8.2 Limitations

There are many areas in which HandWave's recognition accuracy can improve. Some technical and at times physical limitations of the product are as follows:

- **Limited Gesture Vocabulary:** Each system can only learn 8 different specific hand gestures. A library of sign language that the system can recognize is absent. Hence, having a more extensive conversation is not practical.
- **Hardware Bulkiness:** The prototype makes use of an Arduino Nano and a Bluetooth module, both attached to a normal glove. This system is both bulky, and will not be as convenient to use as professional medical wearables.
- **Sensor Calibration and Fatigue:** Flex sensors are made of a mechanical and elastic material that will wear down over time and require frequent calibrating to correct the sensor position as well as to accommodate hands of different sizes. In the long term, sensor fatigue will lead to the signal drifting and will decrease accuracy of gesture recognition.
- **Limitations in Processing:** The glove is not able to learn and improve from use, as the system is not able to learn through Machine Learning (ML). Instead, the system has hard coded threshold values that aid in gesture detection, making the glove inflexible.
- **Environmental Sensitivity:** Current hardware has low tolerance to sudden impacts and moisture. Hence, the hardware is not appropriate for high activity environments, especially outdoors.

8.3 Future Improvement

Although the current “HandWave” prototype establishes a strong base for creating a bilingual assistive technology tool, but several key areas remain for future development to transition the prototype into a commercial-grade product.

Advanced Recognition and Machine Learning:

The transition from a rule- to ML-based system is the most challenging task. This system will use a neural network or a Support Vector Machine (SVM), which will allow the system to:

- **Expand Gesture Vocabulary:** Currently, the system only supports eight signs. With this system, it can encompass hundreds of signs to almost all of Bangla Sign Language (BDSL).
- **User Adaptation:** The system can learn how each user differs in his/her/their finger-bending to balance the system for users who have different hand sizes and mobilities.
- **Dynamic Gesture Recognition:** This system is better at recognizing signs that are dynamic in nature, as opposed to signs that are static.

Hardware Miniaturization and Durability:

A total redesign of the system’s hardware is necessary for added portability and usability enhancements:

- **PCB Implementation:** The custom-built version of the Printed Circuit Board (PCB) will replace the existing breadboard and jumper-wire system to remove size and connection issues.
- **Flexible Electronics:** The use of better ergonomic flex sensors will have an impact on the glove’s mechanical durability, life wear, and lifespan
- **Power Optimization:** An improved power management system, coupled with the newer and better Lithium-polymer battery, will allow the system to be used for longer periods.

Enhanced Connectivity and Features:

Future iterations will focus on increasing the scope of communication:

- **IoT Integration:** Wi-Fi-enabled glove modules will provide additional functionality. This will allow for improved glove management between on-cloud NLP (Natural Language Processing) for improved and integrated dual-layer output.
- **Expanded Sensor Suite:** As the glove progresses, the IMU Sensors (accelerometer, gyroscope and magnetometer) will be added for more advanced hand tracking, to incorporate glove learning, gaming, and various advanced functionality.
- **Mobile App Development:** Building a dedicated smartphone application with customizable settings, gesture logging, and advanced text-to-speech options.
- **Environmental Control:** Smart Home functionality will be expressed in the changing of the glove from purely an informing function to control and manipulate the users’ environment.

References

- [1] M. A. Rabbani et al., “Associated impairments among children with cerebral palsy in rural Bangladesh: Findings from the Bangladesh Cerebral Palsy Register,” *Pediatric Research*, vol. 83, pp. 160–168, 2022.
- [2] “Disability Rights in Bangladesh: Lack of data hampering progress,” *The Daily Star*, 2023.
- [3] “Speech and language delay in children: Prevalence and risk factors,” *Indian J. Pediatric*, vol. 85, pp. 907–913, 2018.
- [4] M. Kamrujjaman, “Oral motor difficulties and speech intelligibility in Bangla speaking children with Down syndrome,” *J. Health & Med. Sciences*, vol. 1, no. 1, pp. 81–91, 2019.
- [5] Y. Jhunjhunwala, P. Shah, P. Patil, and J. Waykule, “Sign language to speech conversion using Arduino,” *Int. J. Res. Innov. Eng. Res.*, vol. 2, no. 3, pp. 55–60, 2017.
- [6] M. I. Hossain, A. Antim, and M. I. Ahmed, “Sign language to speech conversion,” *Int. J. Adv. Res. Electr. Electron. Instrum. Eng.*, vol. 8, no. 5, pp. 1780–1786, 2019.
- [7] A. Kumar and R. Sharma, “Calibration and real-time gesture mapping for sign language gloves,” *IEEE Sensors Lett.*, vol. 4, no. 2, pp. 110–117, 2020.
- [8] P. Patil and J. Waykule, “Gesture recognition using flex sensors and accelerometer for sign-to speech conversion,” *Int. J. Innov. Sci. Res. Technol.*, vol. 3, no. 5, pp. 202–209, 2018.
- [9] R. Ambar et al., “Development of a wearable device for sign language recognition,” *J. Phys. Conf. Ser.*, vol. 1019, no. 1, p. 012110, 2018.
- [10] P. Nandhini, R. Priya, and R. Sridevi, “Hybrid sensor glove for improved sign language recognition,” *Int. J. Adv. Comput. Technol.*, vol. 9, no. 6, pp. 92–98, 2020.
- [11] K. Kudrinko, E. Flavin, X. Zhu, and Q. Li, “Wearable sensor-based sign language recognition: A comprehensive review,” *Sensors*, vol. 20, no. 15, p. 4234, 2020.

- [12] M. Ahmed, T. Chowdhury, and M. Islam, “Comparative study of rule-based and ML-based gesture recognition for sign-to-speech systems,” *J. Embedded Syst. Res.*, vol. 9, no. 2, pp. 34–42, 2021.
- [13] R. Saha, T. Dey, and A. Biswas, “Android-integrated smart glove for sign-to-speech communication,” *Int. J. IoT Smart Devices*, vol. 5, no. 1, pp. 45–52, 2021.
- [14] V. Mishra and D. Singh, “Buffered sign-to-speech translation using Arduino and DFPlayer,” *Procedia Comput. Sci.*, vol. 200, pp. 165–172, 2022.
- [15] T. Rahman, M. Hasan, and R. Khan, “Textile-based stretch sensors for wearable sign language gloves,” *IEEE Access*, vol. 10, pp. 91120–91129, 2022.
- [16] R. Alam, S. Hossain, and F. Akter, “Smart glove-based sign language conversion: Design trends and energy-efficient architectures,” *Int. J. Electron. Commun. Syst.*, vol. 11, no. 4, pp. 22–29, 2023.
- [17] H. Begum, O. Chowdhury, M. S. R. Hridoy, and M. M. Islam, “AI-Based Sensory Glove System to Recognize Bengali Sign Language (BaSL),” *IEEE Access*, vol. 12, pp. 145003–145017, 2024.
- [18] S. Maharjan, S. Shrestha, and S. Fernando, “Sign language detection and translation using smart glove,” *Int. J. Adv. Comput. Sci. Appl.*, vol. 15, no. 1, 2024.
- [19] N. Navin and F. A. Farid, “Bilingual sign language recognition: A YOLOv11-based model for Bangla and English alphabets,” *Sensors*, vol. 25, no. 2, 2025.
- [20] A. J. Abougarair and W. A. Arebi, “Smart glove for sign language translation,” *Int. Rob. Auto. J.*, vol. 8, no. 2, 2022.
- [21] V. S. Janvi et al., “IoT-enabled sign language to speech converter for enhanced communication,” *Int. J. IoT Smart Devices*, vol. 8, no. 2, 2024.
- [22] M. Hassan et al., “A smart glove-based system for dynamic sign language translation using LSTM networks,” *Technologies*, vol. 13, no. 1, 2025.

- [23] S. Wadkar, S. Hekare et al., “Bridging silence: A real-time gesture-to-voice translator using ESP32 and flex sensors,” *Int. J. Adv. Res. Sci. Commun. Technol. (IJARSCT)*, vol. 5, 2025.
- [24] O. Chowdhury, M. Hridoy et al., “Smart gloves for people with speech disability,” in *Proc. IEEE ICCIT*, 2024, pp. 1–6.
- [25] B. Vijaya Kumari, K. Durgesh et al., “IoT-based smart assistance gloves for disabled people,” *Int. J. Eng. Res. Technol. (IJERT)*, vol. 12, no. 5, 2023.
- [26] A. K. Saha, S. Dutta, S. Roy, and S. Bhowmik, “Hand gesture to speech and text conversion device,” *Int. J. Innovative Technol. Exploring Eng. (IJITEE)*, vol. 9, no. 4, pp. 4123–4127, Feb. 2020.
- [27] M. A. Hossain, M. R. Islam, S. A. Rafi, and M. N. Uddin, “Smart glove for sign language recognition and communication,” *J. Intell. Syst.*, vol. 31, no. 1, pp. 1–15, 2022.
- [28] Perplexity AI, “Perplexity AI search engine,” 2025. [Online]. Available: <https://www.perplexity.ai>. [Accessed: Feb. 13, 2026].
- [29] SciSpace, “SciSpace literature review tool,” 2025. [Online]. Available: <https://www.scispace.com>. [Accessed: Feb. 13, 2026].

Appendix

Arduino IDE Code:

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SH110X.h>
#include <SoftwareSerial.h>

// --- OLED setup ---
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SH1106G display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// --- MP3 module setup ---
SoftwareSerial mp3Serial(10, 11); // RX, TX

// --- Bluetooth using hardware UART ---
#define BT Serial // HC-05 connected to D0/D1

// --- Flex sensor pins ---
const int flexPin1 = A1; // I need water
const int flexPin2 = A0; // Thank you
const int flexPin3 = A3; // Hello
const int flexPin4 = A6; // I need food
const int flexPin5 = A2; // Please help me

// --- Thresholds ---
const int TH1 = 800;
const int TH2 = 800;
const int TH3 = 850;
const int TH4 = 800;
const int TH5 = 800;

// Bitmap sizes
#define BMP_W 100
#define BMP_H 24

// -----
// PASTE YOUR EXISTING BITMAP ARRAYS HERE EXACTLY AS THEY ARE:
// waterBitmap
// thankYouBitmap
// helloBitmap
// foodBitmap
```

```
// helpBitmap
// -----

const uint8_t waterBitmap[] PROGMEM = {
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x0f, 0x80, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x1f, 0xc0, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x10, 0x60, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x20, 0x20, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x20, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x20, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0xc0, 0x30, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0xff, 0xff, 0xff, 0xff, 0xc1, 0xe7, 0xff, 0xf8,
0xff, 0xff, 0xff,
0xff, 0xf0, 0xff, 0xff, 0xff, 0xff, 0x83, 0x67, 0xff, 0xf8, 0xff, 0xff, 0xff,
0xff, 0xf0, 0x08,
0x48, 0x84, 0x80, 0x02, 0x30, 0x90, 0x10, 0x20, 0x04, 0x04, 0x10, 0x20, 0x8e,
0x48, 0xc4, 0x81,
0x02, 0x1c, 0x90, 0x10, 0x20, 0x04, 0x04, 0x10, 0x20, 0x8f, 0x08, 0x44, 0x83,
0x83, 0x3c, 0x90,
0x10, 0x22, 0x0c, 0x1f, 0x10, 0x60, 0x8f, 0x48, 0x24, 0x87, 0x03, 0xbc, 0x90,
0x10, 0x26, 0x1c,
0x3f, 0x10, 0xe0, 0x4d, 0x48, 0x24, 0x8c, 0x80, 0xec, 0x93, 0x90, 0x2e, 0x34,
0x35, 0x91, 0xe0,
0x4d, 0x48, 0x24, 0x8c, 0x00, 0xcc, 0x93, 0xd0, 0x2a, 0x24, 0x64, 0x91, 0xa0,
0x4d, 0x48, 0x24,
0x98, 0x00, 0x8c, 0x93, 0xd0, 0x3a, 0x64, 0xc4, 0x93, 0x00, 0x41, 0x48, 0x24,
0x90, 0x00, 0x8c,
0x93, 0x50, 0x32, 0x44, 0x84, 0x12, 0x30, 0x41, 0x48, 0x64, 0x99, 0x00, 0x04,
0x93, 0x70, 0x32,
0x64, 0xc5, 0x13, 0x20, 0x21, 0x48, 0x64, 0x8c, 0x80, 0x04, 0x93, 0x30, 0x22,
0x34, 0x65, 0x91,
0xa0, 0x21, 0x48, 0xf4, 0x84, 0x80, 0x04, 0x92, 0x30, 0x22, 0x14, 0x37, 0x90,
0x80, 0x33, 0xc8,
0xfc, 0x87, 0x00, 0x04, 0x90, 0x30, 0x02, 0x1c, 0x1d, 0x90, 0xe0, 0x1e, 0xc8,
0x6c, 0x83, 0x00,
0x04, 0x90, 0x30, 0x02, 0x0c, 0x1c, 0x10, 0x60, 0x0e, 0x48, 0x0c, 0x89, 0x00,
0x04, 0x90, 0x10,
0x02, 0x2c, 0x0c, 0x11, 0x20, 0x00, 0x48, 0x04, 0x89, 0x00, 0x04, 0x90, 0x10,
```

```

0x02, 0x24, 0x0c,
0x11, 0x20, 0x00, 0x48, 0x04, 0x89, 0x80, 0x04, 0x90, 0x10, 0x02, 0x24, 0x04,
0x11, 0x20, 0x00,
0x08, 0x04, 0x80, 0x00, 0x04, 0x90, 0x10, 0x02, 0x04, 0x04, 0x10, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00
};

```

```

const uint8_t thankYouBitmap[] PROGMEM = {
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x78,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01, 0xfc,
0x7f, 0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xe7, 0xff, 0xff, 0xf0, 0x01, 0xfc, 0x7f, 0xff, 0xff,
0xff, 0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xf0, 0x03, 0xbc, 0x70, 0x00, 0x03, 0x80, 0xe0, 0x00,
0x0c, 0x1e, 0x06,
0x00, 0x00, 0x03, 0x10, 0x70, 0x00, 0x03, 0x81, 0xc0, 0x00, 0x0c, 0x1c, 0x06,
0x00, 0x00, 0x03,
0x80, 0xf0, 0x00, 0x03, 0x81, 0xc0, 0x00, 0x3c, 0x1c, 0x06, 0x00, 0x00, 0x01,
0x87, 0xf0, 0x00,
0x03, 0x81, 0x80, 0x01, 0xfc, 0x1c, 0x06, 0x03, 0xc0, 0x00, 0xff, 0xf0, 0x06,
0x03, 0x81, 0x80,
0x07, 0xfc, 0x1c, 0x06, 0x07, 0xc0, 0x00, 0xfc, 0x70, 0x3f, 0xc3, 0x81, 0x80,
0x1f, 0x0c, 0x1c,
0x06, 0x1f, 0xc0, 0x01, 0xf0, 0x70, 0x7f, 0xe3, 0x81, 0xc0, 0x7c, 0x0c, 0x1c,
0x06, 0x39, 0xc0,
0x03, 0xc0, 0x70, 0x7e, 0xf3, 0x80, 0xc0, 0xf0, 0x0c, 0x1c, 0x06, 0xf1, 0x80,
0x03, 0x80, 0x70,
0x7f, 0x7b, 0x80, 0xc0, 0xe0, 0x0c, 0x1c, 0x07, 0xe3, 0x80, 0x03, 0xc0, 0x70,
0x7f, 0x3b, 0x80,
0xe0, 0xf8, 0x0c, 0x1c, 0x07, 0xc3, 0x80, 0x01, 0xf0, 0x70, 0x7e, 0x1f, 0x80,

```

```

0x60, 0x3e, 0x0c,
0x1c, 0x07, 0x83, 0x80, 0x00, 0x3c, 0x70, 0x3c, 0x0f, 0x80, 0x60, 0x0f, 0x0c,
0x1c, 0x07, 0x03,
0x80, 0x00, 0x1e, 0x70, 0x00, 0x0f, 0x80, 0x60, 0x03, 0xcc, 0x1c, 0x06, 0x03,
0x80, 0x00, 0x07,
0x70, 0x00, 0x07, 0x80, 0x60, 0x01, 0xfc, 0x1c, 0x00, 0x03, 0x80, 0x00, 0x03,
0xf0, 0x00, 0x07,
0x80, 0xe0, 0x00, 0x7c, 0x1c, 0x00, 0x03, 0x80, 0x00, 0x01, 0xf0, 0x00, 0x03,
0x80, 0xe0, 0x00,
0x3c, 0x1c, 0x00, 0x01, 0x80, 0x00, 0x00, 0xf0, 0x00, 0x03, 0x83, 0xc0, 0x00,
0x1c, 0x1c, 0x00,
0x01, 0xc0, 0x00, 0x00, 0x70, 0x00, 0x01, 0x87, 0xc0, 0x00, 0x1c, 0x1c, 0x00,
0x01, 0xc0, 0x00,
0x00, 0x30, 0x00, 0x01, 0x87, 0x00, 0x00, 0x0c, 0x0c, 0x00, 0x00, 0xe0, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x40, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00
};

```

```

const uint8_t helloBitmap[] PROGMEM = {
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x01, 0xff,
0xff, 0xff, 0xff, 0xf1, 0xff, 0xff, 0xff, 0xff, 0xff, 0xfc, 0x70, 0x01, 0xff,
0xff, 0xff, 0xff,
0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf0, 0x00, 0x01, 0xc0, 0x00, 0x3e,
0x1f, 0xc0, 0x7c,
0x00, 0x00, 0x03, 0x87, 0xf0, 0x00, 0x01, 0xc0, 0x00, 0x38, 0x07, 0x00, 0xf0,
0x00, 0x00, 0x03,
0x81, 0xc0, 0x00, 0x01, 0xfc, 0x00, 0x38, 0x07, 0x00, 0xe0, 0x00, 0x00, 0x03,
0x81, 0xc0, 0x00,
0x07, 0xff, 0x80, 0x38, 0x07, 0x01, 0xe0, 0x00, 0xf8, 0x7f, 0x81, 0xc0, 0x00,
0x0f, 0xe7, 0xc0,
0x38, 0x07, 0x01, 0xc0, 0x03, 0xff, 0xff, 0x81, 0xc0, 0x00, 0x0f, 0xc1, 0xe0,

```



```

0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0xff, 0xff, 0xff, 0xff, 0x09, 0x7f, 0xff, 0xfe,
0x3f, 0xff, 0xff,
0xff, 0xf0, 0xff, 0xff, 0xff, 0xff, 0x09, 0x7f, 0xff, 0xfe, 0x3f, 0xff, 0xff,
0xff, 0xf0, 0x08,
0x49, 0x89, 0x02, 0x0f, 0x48, 0x12, 0x04, 0x10, 0x02, 0x00, 0x18, 0x20, 0x8e,
0x49, 0x89, 0x02,
0x0f, 0x48, 0x12, 0x04, 0x10, 0x02, 0x06, 0x10, 0x20, 0x8e, 0x48, 0xc9, 0x06,
0x0d, 0x48, 0x32,
0x0c, 0x11, 0x06, 0x0f, 0x10, 0x60, 0x0f, 0x48, 0x49, 0x0e, 0x01, 0x48, 0x72,
0x1c, 0x11, 0x8e,
0x1f, 0x90, 0xe0, 0x4d, 0x48, 0x49, 0x1a, 0x03, 0x48, 0xd2, 0x34, 0x13, 0x1a,
0x30, 0xd1, 0xa0,
0x4d, 0x48, 0x49, 0x32, 0x06, 0x49, 0x92, 0x64, 0x17, 0x12, 0x30, 0x51, 0x20,
0x49, 0x48, 0x49,
0x22, 0x06, 0x49, 0x12, 0x44, 0x1d, 0x32, 0x64, 0x53, 0x20, 0x41, 0x48, 0x09,
0x22, 0x04, 0x49,
0x12, 0x44, 0x1d, 0x22, 0x40, 0x52, 0x20, 0x41, 0x48, 0xc9, 0x32, 0x04, 0x49,
0x92, 0x64, 0x19,
0x32, 0x65, 0xd3, 0x20, 0x21, 0x48, 0xe9, 0x1a, 0x06, 0x48, 0x92, 0x24, 0x19,
0x1a, 0x31, 0xd1,
0xa0, 0x21, 0x48, 0xe9, 0x0a, 0x03, 0x48, 0x52, 0x34, 0x01, 0x0a, 0x19, 0xd0,
0xe0, 0x13, 0xc8,
0xf9, 0x0e, 0x01, 0xc8, 0x72, 0x1c, 0x01, 0x06, 0x0d, 0x90, 0x60, 0x1e, 0xc8,
0xd9, 0x06, 0x00,
0xc8, 0x32, 0x0c, 0x01, 0x06, 0x0c, 0x10, 0x60, 0x0c, 0xc8, 0x19, 0x16, 0x00,
0xc8, 0x32, 0x6c,
0x01, 0x12, 0x04, 0x11, 0x20, 0x00, 0x48, 0x09, 0x12, 0x00, 0x48, 0x12, 0x2c,
0x01, 0x12, 0x04,
0x11, 0x20, 0x00, 0x48, 0x09, 0x12, 0x00, 0x48, 0x12, 0x04, 0x01, 0x12, 0x04,
0x11, 0x20, 0x00,
0x40, 0x09, 0x02, 0x00, 0x48, 0x12, 0x04, 0x00, 0x80, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
};

```

```

const uint8_t helpBitmap[] PROGMEM = {
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,

```

0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0xff, 0xff, 0xff, 0xff, 0xfc, 0xff, 0xff, 0xff,
0xfe, 0x3f, 0xff,
0xff, 0xf0, 0xff, 0xff, 0xff, 0xff, 0xfc, 0xff, 0xff, 0xff, 0xfe, 0x3f, 0xff,
0xff, 0xf0, 0x18,
0x53, 0x12, 0x60, 0x40, 0x60, 0x44, 0x16, 0x24, 0x01, 0x00, 0x80, 0x10, 0x8c,
0x53, 0x12, 0x40,
0x40, 0x20, 0x42, 0x13, 0x24, 0x01, 0x00, 0x80, 0x10, 0x0e, 0x51, 0x92, 0x40,
0xe0, 0x30, 0x47,
0x11, 0x24, 0x03, 0x81, 0x80, 0x10, 0x0e, 0x50, 0x92, 0x41, 0xf0, 0x10, 0x47,
0x91, 0x20, 0x07,
0xc1, 0xb0, 0x10, 0x4e, 0x50, 0x92, 0x43, 0x50, 0x14, 0x44, 0x90, 0xa4, 0x0d,
0x43, 0xb1, 0x90,
0x4d, 0x50, 0x92, 0x02, 0x48, 0x1e, 0x44, 0x91, 0xa4, 0x09, 0x26, 0xb3, 0x90,
0x49, 0x50, 0x12,
0x06, 0x48, 0x0e, 0x40, 0x93, 0x24, 0x19, 0x24, 0xbb, 0xd0, 0x41, 0x50, 0x12,
0x04, 0x48, 0x0a,
0x40, 0x92, 0x24, 0x11, 0x24, 0x8b, 0xd0, 0x01, 0x50, 0x92, 0x04, 0x48, 0x08,
0x41, 0x92, 0x24,
0x11, 0x24, 0x83, 0x70, 0x21, 0x51, 0x92, 0x06, 0x78, 0x50, 0x4b, 0x13, 0x24,
0x19, 0xe6, 0xd3,
0x30, 0x21, 0x51, 0xd2, 0x03, 0x78, 0x70, 0x4f, 0x13, 0x24, 0x0d, 0xe2, 0xf1,
0x30, 0x23, 0x51,
0xd2, 0x01, 0x78, 0x30, 0x4e, 0x11, 0xa4, 0x05, 0xe3, 0xa0, 0x30, 0x17, 0xd1,
0xb2, 0x41, 0xd0,
0x20, 0x43, 0x10, 0xe4, 0x07, 0x41, 0x80, 0x30, 0x1e, 0xd0, 0x32, 0x40, 0xc0,
0x00, 0x41, 0x90,
0x64, 0x03, 0x01, 0x80, 0x30, 0x0c, 0xd0, 0x32, 0x70, 0xc0, 0x00, 0x40, 0x90,
0x64, 0x03, 0x04,
0x80, 0x30, 0x00, 0xd0, 0x12, 0x70, 0x40, 0x00, 0x40, 0xd0, 0x6c, 0x01, 0x04,
0x80, 0x10, 0x00,
0x50, 0x12, 0x70, 0x40, 0x00, 0x40, 0x50, 0x2c, 0x01, 0x00, 0x80, 0x10, 0x00,
0x40, 0x10, 0x20,

```

0x40, 0x00, 0x00, 0x40, 0x20, 0x01, 0x00, 0x00, 0x10, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00
};

```

```

const uint8_t fineBitmap[] PROGMEM = {
0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x0e,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x30, 0x00, 0x00, 0x00, 0x1f,
0x80, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0xfc, 0x00, 0x00, 0x00, 0x3f, 0xc0, 0x00, 0x00,
0x00, 0x00, 0x00,
    0x00, 0x01, 0xfe, 0x00, 0x00, 0x00, 0x31, 0xe0, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x01, 0xcf,
    0x00, 0x00, 0x00, 0x60, 0xe0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01, 0x83,
0x00, 0x00, 0x00,
    0x60, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01, 0x01, 0x00, 0x00, 0x00,
0x60, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x01, 0x00, 0x00, 0x00, 0x00, 0x60, 0x00, 0x00,
0x00, 0x00, 0x00,
    0x00, 0x00, 0x01, 0x80, 0x00, 0xff, 0xff, 0xff, 0xfc, 0xff, 0xff, 0xff, 0xff,
0xe7, 0xff, 0xff,
    0xff, 0xf0, 0xff, 0xff, 0xff, 0xfc, 0xff, 0xff, 0xff, 0xff, 0xe7, 0xff, 0xff,
0xff, 0xf0, 0x0e,
    0x1f, 0xf7, 0x18, 0x00, 0x1e, 0x60, 0x0d, 0xc0, 0x70, 0xfd, 0x98, 0x00, 0x07,
0x1b, 0x27, 0x18,
    0x00, 0x0c, 0x60, 0x0c, 0xc0, 0x38, 0xd9, 0x98, 0x00, 0x47, 0x9b, 0x21, 0x18,
0x26, 0x0c, 0x41,
    0xbc, 0x83, 0x3c, 0xd9, 0x9f, 0xc0, 0x6f, 0x9b, 0x21, 0x98, 0x67, 0x4c, 0xc3,
0xfc, 0x83, 0x3c,
    0xd9, 0x9f, 0xc0, 0x6e, 0xdb, 0x21, 0x98, 0x67, 0x4c, 0xc3, 0xfc, 0x83, 0x76,
0xd9, 0x9f, 0xe0,
    0x6e, 0xdb, 0x20, 0x98, 0x27, 0xcc, 0xc2, 0x4c, 0x81, 0x36, 0xd9, 0x99, 0x60,
0x26, 0xdb, 0x20,
    0x98, 0x37, 0xec, 0xc2, 0x6c, 0x81, 0xb6, 0xd9, 0x99, 0x60, 0x34, 0xdb, 0x21,
0x98, 0x37, 0xec,
    0xc3, 0xcc, 0x81, 0xb6, 0xd9, 0x9b, 0x60, 0x30, 0xdb, 0x23, 0xd8, 0x33, 0xec,
0xc3, 0x8c, 0x81,
    0x86, 0xd9, 0x9f, 0x60, 0x30, 0xfb, 0x23, 0xf8, 0x18, 0x6c, 0xc3, 0x8c, 0x80,
0xc7, 0xd9, 0x8e,
    0x60, 0x19, 0xfb, 0x23, 0xf8, 0x18, 0x6c, 0xd3, 0x8c, 0x80, 0xcf, 0xd9, 0x86,
0xe0, 0x1f, 0xfb,

```

```

    0x21, 0x78, 0x1c, 0xcc, 0x78, 0x0c, 0x80, 0x7d, 0xd9, 0x81, 0xc0, 0x0f, 0x3b,
    0x20, 0x38, 0x0c,
    0xcc, 0x78, 0x0c, 0x80, 0x7d, 0xd9, 0x87, 0xc0, 0x06, 0x1b, 0x20, 0x18, 0x0f,
    0xcc, 0x78, 0x0c,
    0x80, 0x10, 0xd9, 0x87, 0x80, 0x00, 0x1b, 0x20, 0x18, 0x07, 0x8c, 0x70, 0x0c,
    0x80, 0x00, 0x59,
    0x87, 0xc0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x01, 0xe0, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xe0, 0x00,
    0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x70, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x30, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00,
    0x00, 0x00, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00
};

```

```

const uint8_t happyBitmap[] PROGMEM = {
    0x00, 0x00, 0x00, 0x00, 0x0f, 0x00, 0x00, 0x00, 0x00, 0x00, 0x3c, 0x00, 0x00,
    0x00, 0x00, 0x00,
    0x00, 0x7f, 0xf0, 0x00, 0x00, 0x00, 0x01, 0xff, 0xc0, 0x00, 0x00, 0x00, 0x00,
    0x00, 0xff, 0xfc,
    0x00, 0x00, 0x00, 0x03, 0xff, 0xf0, 0x00, 0x00, 0x00, 0x00, 0x01, 0xf8, 0xfe,
    0x00, 0x00, 0x00,
    0x07, 0xe3, 0xf8, 0x00, 0x00, 0x00, 0x00, 0x01, 0xc0, 0x1e, 0x00, 0x00, 0x00,
    0x07, 0x00, 0x78,
    0x00, 0x00, 0x00, 0x00, 0x01, 0xc0, 0x06, 0x00, 0x00, 0x00, 0x07, 0x00, 0x18,
    0x00, 0x00, 0x00,
    0x00, 0x01, 0xc0, 0x00, 0x00, 0x00, 0x00, 0x07, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x01, 0xc0,
    0x00, 0x00, 0x00, 0x00, 0x07, 0x00, 0x38, 0x00, 0xff, 0xff, 0xff, 0xbf, 0xff,
    0xff, 0xf8, 0x06,
    0x11, 0xff, 0xfc, 0x7d, 0xf0, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf8, 0x0f,
    0x31, 0xff, 0xfe,
    0xff, 0xf0, 0xff, 0xff, 0xff, 0xff, 0xff, 0xff, 0xf8, 0x0f, 0xf9, 0xff, 0xff,
    0xef, 0xf0, 0x00,
    0xe0, 0x1c, 0xf1, 0xc7, 0xc0, 0xe0, 0x0f, 0xf9, 0xc7, 0x07, 0xc7, 0xc0, 0x30,
    0xfe, 0x1c, 0x71,
    0xc1, 0xe0, 0xe0, 0x0f, 0xf9, 0xc7, 0x07, 0xe3, 0xc0, 0x70, 0xff, 0x1c, 0x71,
    0xc0, 0xf0, 0xe0,
    0x07, 0xb9, 0xc7, 0x0f, 0xf3, 0xc0, 0x38, 0xff, 0x9c, 0x71, 0xc0, 0x70, 0xe0,
    0x00, 0x79, 0xc7,
    0x0f, 0xf1, 0xc0, 0x38, 0xfb, 0x9c, 0x71, 0xc0, 0x30, 0xe0, 0x01, 0xf1, 0xc7,
    0x1f, 0xf1, 0xc0,

```

```

    0x3c, 0xfb, 0x9c, 0x71, 0xc0, 0x30, 0xe0, 0x03, 0xe1, 0xc7, 0x1f, 0xf1, 0xc0,
    0x1c, 0xf3, 0x9c,
    0x71, 0xc0, 0x70, 0xe0, 0x03, 0xc1, 0xc7, 0x0e, 0xf1, 0xc0, 0x1e, 0x03, 0x9c,
    0x71, 0xc1, 0xf8,
    0xe0, 0x03, 0xe1, 0xc7, 0x00, 0x01, 0xc0, 0x0e, 0x03, 0xdc, 0x71, 0xc1, 0xfc,
    0xe0, 0x03, 0xf9,
    0xc7, 0x00, 0x01, 0xc0, 0x0f, 0x07, 0xfc, 0x71, 0xc1, 0xff, 0xe0, 0x01, 0xff,
    0xc7, 0x00, 0x01,
    0xc0, 0x07, 0xcf, 0xfc, 0x71, 0xc1, 0xff, 0xe0, 0x00, 0x3f, 0xc7, 0x00, 0x01,
    0xc0, 0x03, 0xff,
    0xfc, 0x71, 0xc0, 0xe7, 0xe0, 0x00, 0x0f, 0xc7, 0x00, 0x01, 0xc0, 0x01, 0xfe,
    0x3c, 0x71, 0xc0,
    0x03, 0xe0, 0x00, 0x07, 0xc7, 0x00, 0x01, 0xc0, 0x00, 0x78, 0x1c, 0x71, 0xc0,
    0x01, 0xe0, 0x00,
    0x03, 0xc7, 0x00, 0x01, 0xc0, 0x00, 0x00, 0x0c, 0x71, 0xc0, 0x00, 0xe0, 0x00,
    0x0f, 0xc7, 0x00,
    0x01, 0xc0, 0x00, 0x00, 0x04, 0x00, 0x00, 0x00, 0x00, 0x40, 0x00, 0x1f, 0xc0, 0x00,
    0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x3f, 0xf0, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x33, 0xf8, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00,
    0x00, 0x33, 0xfc, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x3f, 0x1c,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x3e, 0x08,
    0x00, 0x00, 0x00
};

```

```

const uint8_t notWellBitmap[] PROGMEM = {
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xff, 0xcf, 0xf3, 0xf8, 0xff,
    0xbf, 0xff, 0x4f,
    0xf2, 0x6f, 0xff, 0xff, 0xd0, 0xff, 0xff, 0xff, 0xfc, 0xff, 0xff, 0xff, 0xcf,
    0xfe, 0xff, 0xff,

```

```

    0xff, 0xf0, 0xff, 0xff, 0xff, 0xfc, 0xff, 0xff, 0xff, 0xcf, 0xfe, 0xff, 0xff,
0xff, 0xf0, 0x18,
    0xfe, 0x2e, 0x18, 0x00, 0x6c, 0x05, 0xc0, 0x2e, 0x9c, 0x98, 0x01, 0xb0, 0x4c,
0xd7, 0x24, 0x18,
    0x00, 0x4c, 0x04, 0x80, 0x24, 0x99, 0x88, 0x01, 0x20, 0x4e, 0xd3, 0x24, 0x38,
0x48, 0x4c, 0x7c,
    0x83, 0xe5, 0xd9, 0x8f, 0x81, 0x20, 0x5e, 0xd1, 0x24, 0x78, 0x58, 0x48, 0xfc,
0x87, 0xe4, 0xe9,
    0x8f, 0x8d, 0x20, 0x5a, 0xd1, 0x24, 0xf8, 0x5a, 0x48, 0xfc, 0x87, 0xe4, 0xe9,
0xf, 0xdd, 0x20,
    0x5e, 0xd1, 0x24, 0xd8, 0x5b, 0x48, 0xb4, 0x85, 0xa4, 0x29, 0x0b, 0xdf, 0x20,
0x7b, 0xd1, 0x25,
    0xd8, 0x5f, 0x48, 0x94, 0x84, 0xa4, 0x29, 0x0b, 0xdf, 0x20, 0x6b, 0xd3, 0xa5,
0xd8, 0x7f, 0x48,
    0xf4, 0x86, 0xa4, 0x69, 0x0a, 0xdb, 0x20, 0x23, 0xd3, 0xa5, 0xd8, 0x6d, 0x48,
0xe4, 0x87, 0x24,
    0x49, 0x0e, 0xdb, 0x20, 0x23, 0xd3, 0xe4, 0xf8, 0x2d, 0x48, 0xe4, 0x87, 0x24,
0xc9, 0x0e, 0xcb,
    0x20, 0x37, 0xd3, 0xe4, 0x78, 0x23, 0x4a, 0x44, 0x82, 0x24, 0xc9, 0x44, 0x81,
0x20, 0x3f, 0xd3,
    0xe4, 0x38, 0x33, 0x4e, 0x04, 0x80, 0x24, 0x09, 0xc1, 0x81, 0x20, 0x1e, 0xd0,
0x64, 0xd8, 0x36,
    0x4e, 0x04, 0x80, 0x24, 0x09, 0xc7, 0x81, 0x20, 0x0c, 0xd0, 0x24, 0xd8, 0x1e,
0x4e, 0x04, 0x80,
    0x24, 0x08, 0xc7, 0x01, 0x20, 0x00, 0x50, 0x24, 0xc8, 0x1e, 0x46, 0x04, 0x80,
0x24, 0x08, 0xc7,
    0x80, 0x20, 0x00, 0x40, 0x20, 0x48, 0x0c, 0x02, 0x00, 0x00, 0x00, 0x00, 0x43,
0x80, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01, 0xc0, 0x00, 0x00,
0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0xc0, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
    0x00, 0x00, 0x00, 0x00, 0x60, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
    0x00, 0x60, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00
};

```

```
// -----
```

```
// MP3 command function
```

```

void sendMP3Command(byte command, byte param1, byte param2) {
    byte buf[10] = {0x7E, 0xFF, 0x06, command, 0x00, param1, param2, 0xEF};
    for (int i = 0; i < 8; i++) {
        mp3Serial.write(buf[i]);
    }
}

```

```

}

// Display English + Bangla function
void showMessage(const char* englishText, const uint8_t* banglaBmp, int bmpW, int
bmpH) {
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SH110X_WHITE);

    int16_t x1, y1;
    uint16_t w, h;
    display.getTextBounds(englishText, 0, 0, &x1, &y1, &w, &h);

    int enX = (SCREEN_WIDTH - w) / 2;
    display.setCursor(enX, 0);
    display.print(englishText);

    int bmpX = (SCREEN_WIDTH - bmpW) / 2;
    int bmpY = 22;
    display.drawBitmap(bmpX, bmpY, banglaBmp, bmpW, bmpH, SH110X_WHITE);

    display.display();
}

// Display English only
void showEnglishOnly(const char* text) {
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SH110X_WHITE);

    int16_t x1, y1;
    uint16_t w, h;
    display.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);

    int textX = (SCREEN_WIDTH - w) / 2;
    display.setCursor(textX, 20);
    display.print(text);
    display.display();
}

// Display phone messages
void showPhoneMessage(const String &msg) {
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(SH110X_WHITE);

```

```

display.setCursor(0, 0);
display.print("Phone:");

int16_t x1, y1;
uint16_t w, h;
display.getTextBounds(msg.c_str(), 0, 0, &x1, &y1, &w, &h);

int msgX = (SCREEN_WIDTH - w) / 2;
display.setCursor(msgX, 20);
display.print(msg);
display.display();
}

void setup() {
  Serial.begin(9600);      // Bluetooth on D0/D1
  mp3Serial.begin(9600);  //

  Wire.begin();
  if (!display.begin(0x3C, true)) {
    while (1);
  }

  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SH110X_WHITE);
  display.setCursor(20, 25);
  display.print("System Ready");
  display.display();
  delay(1000);
}

void loop() {
  // --- Check Bluetooth first ---
  if (BT.available()) {
    String msg = "";
    while (BT.available()) {
      char c = BT.read();
      if (c != '\n' && c != '\r') {
        msg += c;
      }
    }
  }

  if (msg.length() > 0) {
    showPhoneMessage(msg);
  }
}

```

```

        delay(2000);
    }
}

// --- Read flex sensors ---
int v1 = analogRead(flexPin1);
int v2 = analogRead(flexPin2);
int v3 = analogRead(flexPin3);
int v4 = analogRead(flexPin4);
int v5 = analogRead(flexPin5);

bool f1 = (v1 > TH1);
bool f2 = (v2 > TH2);
bool f3 = (v3 > TH3);
bool f4 = (v4 > TH4);
bool f5 = (v5 > TH5);

// --- COMBINATION checks first ---
if (f1 && f2) {
    showMessage("I am fine", fineBitmap, BMP_W, BMP_H);
    BT.println("I am fine");
    sendMP3Command(0x03, 0x00, 7);
    delay(1000);
}
else if (f1 && f3) {
    showMessage("I am happy", happyBitmap, BMP_W, BMP_H);
    BT.println("I am happy");
    sendMP3Command(0x03, 0x00, 6);
    delay(1000);
}
else if (f1 && f4) { // A1 + A6
    showMessage("I am not feeling well", notWellBitmap, BMP_W, BMP_H);
    BT.println("I am not feeling well");
    sendMP3Command(0x03, 0x00, 8);
    delay(1000);
}

// --- Single sensor checks ---
else if (f1) {
    showMessage("I need water", waterBitmap, BMP_W, BMP_H);
    BT.println("I need water");
    sendMP3Command(0x03, 0x00, 1);
    delay(1000);
}
else if (f2) {

```

```

    showMessage("Thank you", thankYouBitmap, BMP_W, BMP_H);
    BT.println("Thank you");
    sendMP3Command(0x03, 0x00, 2);
    delay(1000);
}
else if (f3) {
    showMessage("Hello", helloBitmap, BMP_W, BMP_H);
    BT.println("Hello");
    sendMP3Command(0x03, 0x00, 4);
    delay(1000);
}
else if (f4) {
    showMessage("I need food", foodBitmap, BMP_W, BMP_H);
    BT.println("I need food");
    sendMP3Command(0x03, 0x00, 5);
    delay(1000);
}
else if (f5) {
    showMessage("Please help me", helpBitmap, BMP_W, BMP_H);
    BT.println("Please help me");
    sendMP3Command(0x03, 0x00, 3);
    delay(1000);
}

delay(300);
}

```